

ADAPTIVE ACCESSIBILITY SYSTEM FOR LAPTOP INTERACTION

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING



By

Batch - B15

S. Sridevi (21JG1A05A9)

S. Varshini (21JG1A05A7)

P. Sandhya (21JG1A0595)

N. S. Ashrita (21JG1A0587)

Under the esteemed guidance of

Mrs. M. Aswini

Assistant Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN(A)

[Approved by AICTE NEW DELHI, Affiliated to JNTUK Kakinada]

[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi, Madhurawada, Visakhapatnam – 530048

2021 – 2025

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report titled “**ADAPTIVE ACCESSIBILITY SYSTEM FOR LAPTOP INTERACTION**” is a bonafide work of the following IV/IV B.Tech students in the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women affiliated to JNT University, Kakinada during the academic year 2024-25, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology of this University.

S. Sridevi (21JG1A05A9)

S. Varshini (21JG1A05A7)

P. Sandhya (21JG1A0595)

N. S. Ashrita (21JG1A0587)

Signature of the Guide

Mrs. M. Aswini

Assistant Professor

Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba

Professor

Dept. of CSE

External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible and whose constant guidance and encouragement crown all the efforts with success.

We feel elated to extend our sincere gratitude to our guide, **Mrs. M. Aswini**, Assistant Professor for encouragement all the way during analysis of the project. Her annotations, insinuations and criticisms are the key behind the successful completion of the thesis and for providing us all the required facilities.

We would like to take this opportunity to extend our gratitude to Project Coordinator, **Mrs. Y. Sowmya**, Assistant Professor of Computer Science and Engineering for making us to follow a defined path and for advising us to get better improvements in the project.

We express our deep sense of gratitude and thanks to **Dr. P. V. S. Lakshmi Jagadamba**, Professor and Head of the Department of Computer Science and Engineering for her guidance and for expressing her valuable and grateful opinions in the project for its development and for providing lab sessions and extra hours to complete the project.

We would like to take this opportunity to express our profound sense of gratitude to **Dr. G. Sudheer**, Vice Principal and **Dr. R. K. Goswami**, Principal for allowing us to utilize the college resources thereby facilitating the successful completion of our thesis.

We are also thankful to both teaching and non-teaching faculty of the Department of Computer Science and Engineering for giving valuable suggestions for our project.

S. Sridevi (21JG1A05A9)

S. Varshini (21JG1A05A7)

P. Sandhya (21JG1A0595)

N. S. Ashrita (21JG1A0587)

TABLE OF CONTENTS

TOPICS	PAGE NO.
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF SCREENS	iii
1. INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Definition	2
1.3 Objective of the Project	2
1.4 Limitations of the Project	2
1.5 Organization of the Project	3
2. LITERATURE SURVEY	4
2.1 Existing System	11
2.2 Limitations of Existing System	12
2.3 Proposed System	12
3. REQUIREMENT ANALYSIS	14
3.1 Introduction	14
3.2 Software Requirement Specification	14
3.2.1 Software Requirements	14
3.2.2 Hardware Requirements	17
3.3 Non-Functional Requirements	17
3.4 Flowchart of our Project	18
3.5 Algorithms	20

TOPICS	PAGE NO.
4. DESIGN	24
4.1 Introduction	24
4.2 UML Diagrams	25
4.2.1 Use Case Diagram	25
4.2.2 Class Diagram	27
4.2.3 Sequence Diagram	28
4.2.4 Activity Diagram	29
4.3 Module Design	30
4.3.1 System Module	30
4.4 Conclusion	32
5. IMPLEMENTATION AND RESULTS	33
5.1 Introduction	33
5.2 Software Design	33
5.2.1 Front end Design	33
5.2.2 Back-end Design	34
5.3 Implementation	36
5.4 Output Screens	59
5.5 Conclusion	63
6. CONCLUSION	64
7. FUTURE SCOPE	65
8. REFERENCES	66

ABSTRACT

In today's rapidly evolving world of human-computer interaction, traditional tools like the mouse and keyboard, while effective, can sometimes feel restrictive, especially for individuals with physical disabilities or in situations where hands-free control is more practical. This project introduces a creative and accessible solution that replaces these physical devices with a virtual mouse and keyboard controlled entirely by hand gestures and voice commands. Using MediaPipe for real-time hand tracking and gesture recognition, along with TensorFlow for voice command classification, the system allows users to move the cursor, perform clicks, and simulate keyboard actions effortlessly. A webcam captures hand movements, interpreting gestures such as finger counts or directional swipes, while a microphone listens for voice inputs to trigger specific functions. The software is designed with a clear structure, focusing on gesture control for mouse actions and voice control for keyboard simulation. The front end is built using React.js to provide a smooth and interactive user experience, while Python scripts work in the background to connect with the camera and microphone and process the inputs in real time. This system is particularly useful in assistive technology, smart workspaces, and hands-free control applications, aiming to make computer interaction more intuitive, inclusive, and aligned with the capabilities of modern AI.

Keywords: Gesture Recognition, Speech Recognition, Virtual Mouse Control, Computer Vision, MediaPipe, PyAutoGUI, Google Speech-to-Text API, SBERT (Sentence-BERT)

LIST OF FIGURES

S. No.	Figure No.	Figure Name	Page No.
1.	Fig. 1	Existing System	11
2.	Fig. 2	Proposed System	13
3.	Fig. 3	System Architecture	19
4.	Fig. 4	Architecture of SBERT	21
5.	Fig. 5	Use Case Diagram 1	26
6.	Fig. 6	Use Case Diagram 2	26
7.	Fig. 7	Class Diagram	27
8.	Fig. 8	Sequence Diagram	28
9.	Fig. 9	Activity Diagram	29

LIST OF TABLES

S. No.	Table No.	Table Name	Page No.
1.	Table 1	Literature Survey	8
2.	Table 2	Software Requirements	15
3.	Table 3	Python Packages	15
4.	Table 4	Harrrdware Requirements	17

LIST OF SCREENS

S. No.	Screen No.	Screen Name	Page No.
1.	Screen 1	Home page	59
2.	Screen 2	Gesture controller started	59
3.	Screen 3	About page	60
4.	Screen 4	How to use	60
5.	Screen 5	Voice commands	61
6.	Screen 6	Hand gestures	62
7.	Screen 7	Detected gestures	62
8.	Screen 8	Detected Voice Commands	63

1. INTRODUCTION

In today's world, computers have become an essential part of daily life, and people usually interact with them using a keyboard and mouse. The Adaptive Accessibility System introduces a new way to control a computer without needing these traditional tools. Instead, it replaces the mouse with hand gestures and the keyboard with speech-to-text conversion. With the help of a tool called MediaPipe, the system can detect and understand hand gestures to move the cursor and perform clicks. It also uses speech recognition to convert spoken words into text, eliminating the need for typing.

This system is especially helpful for people who may find it difficult to use a keyboard or mouse. It makes interacting with technology easier and more comfortable for everyone. MediaPipe is an open-source framework used to build pipelines for performing computer vision tasks on various types of sensory data, such as video or audio. This powerful tool helps the system track hand movements accurately and in real time. For speech-to-text conversion, the system uses Google's Speech-to-Text API along with Natural Language Processing (NLP) techniques to accurately understand and convert spoken language into written text. The system has a wide range of applications, including education, gaming, smart homes, and hygiene-sensitive environments like hospitals. By combining gestures and voice input, it offers a smooth and efficient way to interact with computers.

1.1 MOTIVATION

The main goal of this project is to make it easier and more comfortable for people to use computers without needing a keyboard or mouse. In many situations, such as in classrooms, smart homes, hospitals, or while gaming, traditional input methods may not always be practical or convenient. This system introduces a new way to control a computer through hands-free interaction, allowing users to perform tasks more naturally and efficiently. It is especially helpful in environments where hygiene, accessibility, or ease of use is important. By offering an alternative to physical input devices, the system creates a smoother, more modern, and inclusive way for users to interact with technology.

1.2 PROBLEM DEFINITION

Most computers today are operated using a mouse and keyboard. While this works well for many users, it can be difficult for those who have trouble using their hands or for people in environments where touching devices is not ideal, such as hospitals or cleanrooms. Traditional input methods are not always suitable or inclusive for everyone.

Because of this, there is a need for a more accessible and touch-free way to interact with computers. The goal is to develop a system that allows users to control a computer using hand gestures and voice commands instead of physical devices. This system should be easy to use, accurate, and useful for people in various situations, including those with limited mobility or in hygiene-sensitive environments.

1.3 OBJECTIVE OF THE PROJECT

The main objective of this project is to develop a system that allows users to control a computer without using traditional input devices like a mouse and keyboard. Instead, the system will use hand gestures for moving the cursor and performing actions, and speech recognition to convert spoken words into text. This will make it easier for people to interact with computers in a more natural and comfortable way.

Another key goal is to improve accessibility for users who may find it difficult to use standard devices due to physical limitations or environmental conditions. The system is also intended to be helpful in hygiene-sensitive environments like hospitals or cleanrooms, where touchless interaction is preferred. Overall, the aim is to create a smooth, simple, and user-friendly experience that works well for different types of users in various settings.

1.4 LIMITATIONS OF THE PROJECT

One of the main limitations of this project is that the system may not work well in low-light or cluttered environments, as hand gesture tracking relies on a clear view of the user's hands. Any obstacles or poor lighting conditions can affect the accuracy of gesture recognition. Also,

fast or unclear hand movements might not be detected properly, which can lead to delays or incorrect actions.

Another limitation is related to voice control. The system might have trouble understanding speech in noisy environments or when the speaker has a strong accent or unclear pronunciation. It also depends on a stable internet connection if cloud-based voice recognition services are used. These factors can reduce the overall performance and reliability of the system in real-world situations.

1.5 ORGANIZATION OF THE PROJECT

Concise overview of rest of the documentation work is explained below:

Chapter 1: Introduction describes the motivation and objective of this project.

Chapter 2: Literature Survey describes the primary terms involved in the development of the Project.

Chapter 3: Analysis deals with the detailed analysis of the project. Software requirements specification further contains software requirements, hardware requirements.

Chapter 4: Contains design part and UML diagrams.

Chapter 5: Contains step by step process and screenshots of the outputs.

Chapter 6: Contains conclusion of the project.

Chapter 7: Contains future scope of the project.

Chapter 8: Contains references.

2. LITERATURE SURVEY

David Richard Tom Hax et al. [1] proposed a hybrid deep learning model to enhance dynamic hand gesture recognition for human-computer interaction. The system targets six gestures: scroll-left, scroll-right, scroll-up, scroll-down, zoom-in, and zoom-out. It combines Inception-v3 for spatial feature extraction with LSTM layers to capture temporal dynamics from video sequences. A diverse dataset with varying lighting and backgrounds was used to ensure robustness. Preprocessing methods like frame segmentation and normalization helped maintain input consistency. The model achieved an average accuracy of 83.66%, with strong performance on zoom gestures. The study emphasizes deep learning's value in gesture-based systems and suggests expanding gesture sets, improving real-time performance, handling occlusions, and applying the model in accessibility tools and immersive virtual environments.

CH. Akash Reddy et al. [2] from Tirumala Engineering College proposed an accessibility system that enables laptop control without a physical mouse or keyboard, aiming to assist users with physical disabilities. The system uses computer vision and real-time video input from a webcam or built-in camera, applying hand-tracking frameworks like MediaPipe to detect and interpret hand gestures. These gestures are mapped to common mouse functions such as cursor movement, clicking, and scrolling. The solution is lightweight, requiring only a standard camera and a dongle, eliminating the need for wearable devices. Designed for ease of use, the system supports future expansion, including voice control and gesture-based shortcuts. It promotes hands-free interaction, enhancing digital accessibility in assistive technology and smart computing environments

Manisha Kasar et al. [3] proposed a real-time hand gesture-controlled virtual mouse system using AI and computer vision. The system employs a webcam or built-in camera to capture hand gestures, which are processed to perform mouse functions like cursor movement, clicks, scrolling, and volume control. Utilizing MediaPipe for hand tracking and PyAutoGUI for cursor control, the model achieves high accuracy even in challenging environments. This approach offers an alternative for users with limited space or hand impairments, reducing reliance on traditional input devices.

Mrs. G. Pavani et al. [4] from JB Institute of Engineering & Technology, Hyderabad, developed an accessibility system that enables laptop control without a physical mouse, using hand gestures as an intuitive input method. The system captures hand movements through a

standard webcam and interprets them using computer vision techniques. Convolutional Neural Networks (CNN), implemented with Keras and TensorFlow, are used to improve recognition accuracy. The solution is designed to perform reliably under varied lighting and background conditions and does not require additional hardware. It provides a hands-free, user-friendly interface aimed at assisting users with physical disabilities. Future enhancements include expanding the gesture vocabulary, integrating voice control, and optimizing real-time processing to enhance usability and adaptability.

Abu Saleh Musa Miah et al. [5] proposed a dynamic hand gesture recognition system that integrates a Multi-Branch Attention Graph (MBAG) network with general deep learning models. This hybrid architecture effectively captures both spatial and temporal features from skeleton-based gesture data. The model was evaluated on standard benchmark datasets like DHG-14/28 and SHREC, demonstrating improved accuracy over existing methods. The attention mechanism within the MBAG enhances the model's ability to focus on relevant spatial and temporal features. However, the system relies heavily on accurate skeleton estimation, which may be affected in cluttered or noisy environments.

G. Park, V. K. Chandrasegar et al. [6] introduced a novel approach to enhance hand gesture recognition accuracy by integrating 2D Fast Fourier Transform (2D-FFT) with Convolutional Neural Networks (CNNs). Utilizing a custom dataset acquired through Impulse Radio Ultra-Wideband (IR-UWB) radar, the study captured various hand gestures, which were then transformed into the frequency domain using 2D-FFT. These transformed images served as input to CNN models for classification. The proposed method demonstrated superior recognition accuracy, reaching up to 95.60%, and required less training time compared to established models like GoogLeNet and ResNet-50. However, the reliance on specialized IR-UWB radar hardware may pose challenges for widespread adoption, and environmental factors could affect the system's robustness in real-world scenarios.

David González León et al. [7] (2022) proposed a hand gesture recognition system utilizing depth video data and a lightweight Convolutional Neural Network (CNN) architecture. The system was designed to operate efficiently on resource-constrained devices, making it suitable for real-time applications. A custom dataset was collected using a depth camera, capturing various dynamic hand gestures. The lightweight CNN was optimized to process depth video inputs, achieving high recognition accuracy while maintaining low computational requirements. This approach enables effective gesture recognition in real-time

scenarios, particularly in environments where computational resources are limited. However, the system's performance may be influenced by varying lighting conditions and backgrounds, and its ability to generalize to unseen gestures or users may be limited.

T.L. Dang et al. [8] proposed an improved hand gesture recognition system that combines hand key points and bounding box features. Their model uses a two-stream architecture: one stream extracts hand key points using a pose estimation network, while the other processes the hand's bounding box with a CNN. By fusing both features, the model achieves enhanced recognition accuracy. The system was trained and evaluated on benchmark datasets like HANDS, OUHANDS, and SHAPE, and it achieved an accuracy of up to 98% with only 0.22 million parameters, making it suitable for real-time and resource-constrained applications. However, the system may face limitations when recognizing unseen gestures and under conditions with partial occlusions.

S. Jeeru et al. [9] introduced a comprehensive RGB-D hand gesture dataset aimed at enhancing gesture recognition systems. Utilizing the Intel RealSense Depth Camera D435, they captured 662 sequences, each comprising 40 frames, encompassing six dynamic gestures: scroll-right, scroll-left, scroll-up, scroll-down, zoom-in, and zoom-out. The dataset includes both RGB and depth frames, recorded under complex backgrounds to simulate real-world conditions. This resource serves as a valuable benchmark for developing and evaluating machine learning models in gesture recognition. However, its scope is confined to six specific gestures, which may limit its applicability in more diverse gesture-based applications.

Y. Shi et al. [10] conducted a comprehensive survey on dynamic gesture recognition, focusing on video-based methods utilizing deep learning. The paper categorizes existing approaches into three main types: two-stream convolutional neural networks (CNNs), 3D CNNs, and Long Short-Term Memory (LSTM) networks. It emphasizes the importance of spatiotemporal feature extraction in capturing the dynamic aspects of gestures. The authors discuss the advantages and limitations of each approach and highlight the challenges in the field, such as variability in hand shapes, occlusions, and real-time processing constraints. The survey serves as a valuable resource for researchers by summarizing the state-of-the-art techniques and suggesting potential directions for future research to develop more robust and natural gesture-based interaction systems.

Dardina Tasmere et al. [11] proposed a real-time hand gesture recognition system using CNN on depth images. The system classifies 11 static gestures and is intended to assist deaf

individuals in communication. Depth data improves performance under varied lighting. The model achieves around 94.61% accuracy. However, it is limited to static gestures and may not perform well with gesture variation or dynamic movement.

Md Abdur Rahim et al. [12] proposed a hand gesture recognition system that utilizes optimal segmentation techniques to enhance human-computer interaction. The system focuses on accurately segmenting hand regions from images or video frames, which is crucial for reliable gesture recognition. By applying advanced segmentation algorithms, the authors aim to isolate hand gestures effectively, even in complex backgrounds. The approach is evaluated using relevant datasets, demonstrating improved accuracy in gesture recognition tasks. However, the system's performance may be affected by variations in lighting conditions and the presence of occlusions, which can impact the segmentation quality.

Table 1. Literature Survey

Ref	Title & Authors	Published Journal & Year	Dataset Used	Approach	Outcome	Limitations
[1]	A Novel Hybrid Deep Learning Architecture for Dynamic Hand Gesture Recognition - David Richard Tom Hax, Pascal Penava, Samira Krodell, Liliya Razova, Richardo Buettner	IEEE Access, 2024	DepthCamera Dataset (6 hand gestures, realistic environment)	Hybrid deep learning model combining Inception-v3 CNN for feature extraction and LSTM for sequence classification.	Achieved 83.66% average accuracy in dynamic hand gesture recognition in realistic environments.	Struggles with similar gestures (e.g., zoom-in vs. zoom-out), requires computational resources, and lacks diverse datasets for broader generalization.
[2]	Accessibility System to Control Laptop Without Mouse or Keyboard - Ch. Akash Reddy, A. Sruthi, G.Hareesh Babu, A. Balaji	International Journal of Scientific Research in Engineering and Management (IJSREM), April 2024	uses webcam-captured images for hand gesture recognition	Computer Vision and Machine Learning (MediaPipe, OpenCV).	Limited to basic actions like selecting, moving, and Scrolling.	Relies on webcam quality for gesture recognition, difficulty in detecting colored objects in the background.
[3]	AI-based Real-Time Hand Gesture-Controlled Virtual Mouse – M. Kasar et al.	Australian Journal of Electrical & Electronics Engineering, 2024	Real-time webcam/live video input (no pre-existing dataset)	AI + computer vision (MediaPipe, PyAutoGUI) for gesture-based mouse control	High accuracy in gesture recognition; suitable for users with hand impairments	Performance may vary with lighting conditions and camera quality
[4]	Accessibility System to Control Laptop Without	International Journal of	The dataset consists of hand	A Convolutional Neural Network	Achieved 90% accuracy. Performs	Performance highly dependent on training data

	Mouse - Mrs. G. Pavani, B. Priyanka, Ch. Keerthana, P. Niharika, V. Prathyusha	Scientific Research in Science, Engineering and Technology (IJSRSET), March–April 2023	gesture images used for training the model	(CNN) is utilized for gesture recognition, while Python, TensorFlow, and MediaPipe are employed for real-time detection.	basic mouse functions (click, scroll, drag).	and lighting conditions. System requires moderate computational resources.
[5]	Dynamic Hand Gesture Recognition Using Multi-Branch Attention Based Graph and General Deep Learning Model – A. S. M. Miah, M. A. M. Hasan, J. Shin	IEEE Access, 2023	DHG-14/28, SHREC	Multi-Branch Attention Graph + CNN-based architecture	Achieved high accuracy on benchmark datasets	Sensitive to skeleton data quality and background noise
[6]	Accuracy Enhancement of Hand Gesture Recognition Using CNN – G. Park, V. K. Chandrasegar, J. Koh	IEEE Access, 2023	Custom dataset collected using IR-UWB radar capturing various hand gestures	Employed Convolutional Neural Networks (CNNs) to improve hand gesture recognition accuracy	Achieved enhanced recognition accuracy compared to previous methods	Dependence on specialized IR-UWB radar hardware; potential challenges in real-world deployment due to environmental noise and hardware constraints
[7]	Video Hand Gestures Recognition Using Depth Camera and Lightweight CNN – D. González León et al.	IEEE Sensors Journal, 2022	Custom depth video dataset (Intel RealSense)	Lightweight CNN for dynamic gesture recognition	High accuracy, real-time performance	Affected by lighting, limited to known gestures
[8]	An Improved Hand Gesture Recognition System Using Keypoints and Hand Bounding Boxes – T.L. Dang et al.	Array, 2022	HANDS, OUHANDS, SHAPE	Two-pipeline architecture using keypoints and bounding boxes	Achieved up to 98% accuracy; lightweight model with 0.22M parameters	May struggle with unseen gestures; performance affected by occlusions
[9]	Depth Camera Based	Data in Brief, 2022	Custom RGB-D dataset (Intel	Captured 662 sequences (40 frames	Provides a robust dataset for training	Limited to six predefined gestures; may require

	Dataset of Hand Gestures – S. Jeeru et al.		RealSense D435) with 6 dynamic gestures	each) of gestures under complex backgrounds	gesture recognition models	expansion for broader applications
[10]	Review of Dynamic Gesture Recognition – Y. Shi et al.	Visual Informatics, 2021	Various public datasets (e.g., Chalearn, NTU RGB+D, SHREC)	Survey of 2D/3D CNNs, LSTM, and hybrid models	Summarizes techniques and trends in dynamic gesture recognition	Limited to existing literature; does not propose or test new models
[11]	Real Time Hand Gesture Recognition in Depth Image using CNN – D. Tasmere, B. Ahmed, S. R. Das	International Journal of Computer Applications, Vol. 174, No. 16, January 2021	Depth images for 11 static gestures	CNN on depth images	94.61% accuracy in real-time static gesture recognition	Limited to static gestures; performance affected by gesture variation
[12]	Hand Gesture Recognition Based on Optimal Segmentation in Human-Computer Interaction – Md Abdur Rahim et al.	IEEE Conference Proceedings, 2021		Optimal segmentation techniques for hand region extraction	Improved accuracy in gesture recognition tasks	Performance may degrade under varying lighting conditions and occlusions

2.1 EXISTING SYSTEM

Based on our literature survey, various systems have been proposed for dynamic hand gesture recognition using deep learning and computer vision techniques. One of the existing systems utilizes a hybrid deep learning approach that combines Inception-v3 (CNN) and RNN with LSTM layers to recognize gestures from video sequences. These systems are specifically designed to improve interaction accuracy in real-world environments by effectively capturing both spatial and temporal features of dynamic gestures.

The primary uses of such systems include:

- **Improved Gesture Recognition Accuracy:** By combining spatial feature extraction (via Inception-v3) with sequential data processing (via LSTM), the system significantly improves the accuracy of recognizing dynamic hand gestures such as scroll-left, scroll-right, scroll-up, scroll-down, zoom-in, and zoom-out.
- **Handling Real-World Conditions:** These models are trained and tested in realistic settings, where lighting, background clutter, and hand variation can affect performance. The hybrid model is more robust in handling these challenges, achieving an average accuracy of 83.66%.
- **Advancing Human-Computer Interaction:** The system supports more natural and intuitive user interfaces by enabling gesture-based control in applications like virtual reality, gaming, smart device interaction, and touchless interfaces.

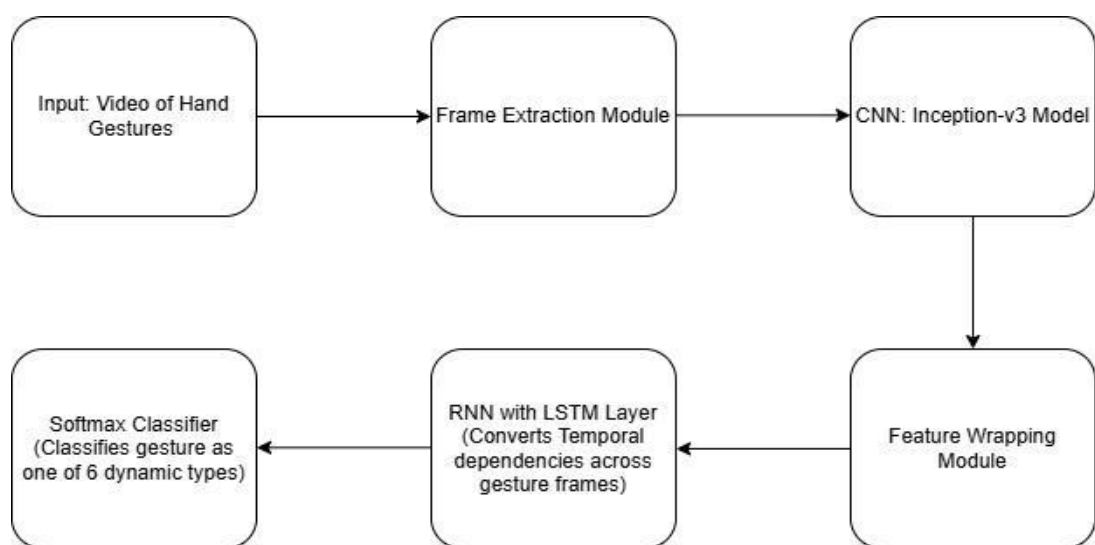


Fig. 1. Existing System

2.2 LIMITATIONS OF EXISTING SYSTEM

- **Limited Dataset Diversity:** The model was trained on a dataset comprising only six specific dynamic hand gestures (scroll-left, scroll-right, scroll-up, scroll-down, zoom-in, and zoom-out). This narrow focus restricts the system's applicability to a broader range of gestures commonly used in real-world human-computer interaction scenarios.
- **Challenges in Realistic Environments:** While the study aimed to address gesture recognition in realistic settings, the model's performance may degrade in more complex real-world situations. For instance, variations in lighting conditions, backgrounds, and the presence of multiple individuals performing gestures simultaneously can adversely affect accuracy.
- **High Computational Requirements:** The hybrid architecture combining Inception-v3 and LSTM networks demands significant computational resources. This high complexity can hinder real-time deployment, especially on devices with limited processing capabilities.

2.3 PROPOSED SYSTEM

The proposed system introduces an adaptive accessibility solution that aims to replace traditional input devices like the mouse and keyboard by leveraging hand gestures and voice input. This system captures live video feed to detect and track hand gestures in real-time, allowing users to perform mouse actions such as moving the cursor, clicking, or dragging. It utilizes MediaPipe's Hand Tracking Model, which combines BlazePalm for efficient hand detection and the Hand Landmark Model for precise key point tracking. Alongside this, the system integrates Google's Speech-to-Text API through a speech recognition model to convert spoken words into text, effectively replacing the need for physical typing. By combining gesture-based control and voice input, this solution enhances accessibility and provides a seamless, touch-free interaction experience particularly beneficial for users with physical impairments or in environments where hands-free operation is preferred.

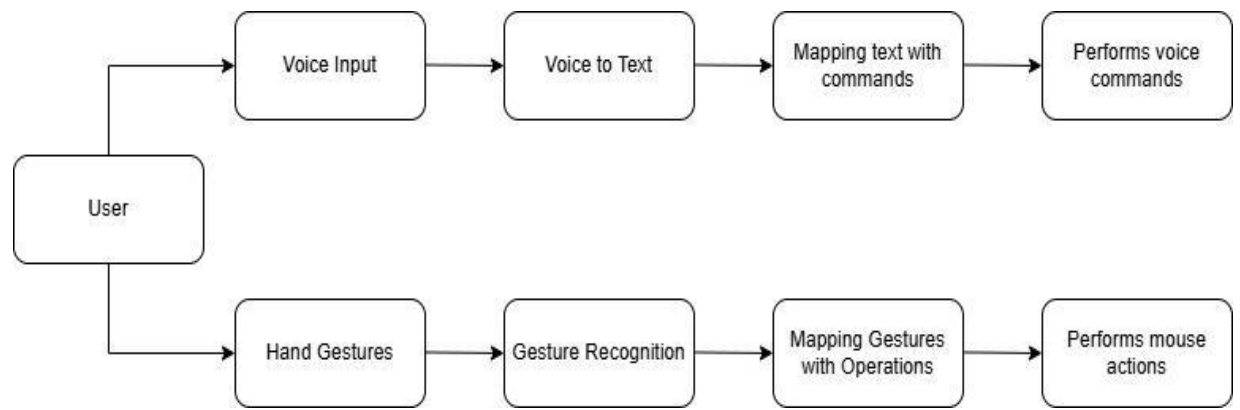


Fig. 2. Proposed System

3. REQUIREMENT ANALYSIS

3.1 INTRODUCTION

The initial part of software development is requirement analysis. The primary goal of this phase is to define the problem and the system that will be developed. The later stages are completely dependent on this phase, hence the system analyst must be more specific and precise about it. Any irregularity in this step will cause complications in the subsequent phases. The main activities include:

- Software Requirements Specification
- Creating Content diagram
- Flowchart of the code

3.2 SOFTWARE REQUIREMENT SPECIFICATION

3.2.1 Software Requirements

The software requirements include the languages, libraries, packages and operating system, and different tools for developing the project. We used python for coding Refer table 1 and table 2 for software requirements.

Minimum Software Requirements

Table 2. Software Requirements

Software	Version
Visual Studio Code Editor	1.99.1
Jupyter Notebook	7.3.2
Python	3.11.9
Operating System	64-bit OS, Windows 11, Version 23H2

Python packages

Python packages we used in our project includes the following table 2.

We used the command pip install “*package_name==version*” to download the packages.

Table 3. Python packages

S.No.	Package Used	Description
1.	cv2 (OpenCV)	Computer vision library used for video processing
2.	mediapipe	Framework for building multimodal ML pipelines (e.g., hands)
3.	pyautogui	GUI automation tool for mouse and keyboard control
4.	math	Standard library for mathematical operations
5.	enum	Provides support for enumerations (Enums)

6.	ctypes	Used to call C functions and manipulate pointers in Python
7.	comtypes	COM client support used with pycaw
8.	pycaw	Python Core Audio Windows Library for controlling volume
9.	google.protobuf	Used for converting Mediapipe protobufs to dicts
10.	screen_brightness_control	Used to get and set screen brightness levels
11.	sys	Provides access to system specific parameters and functions
12.	os	Interacts with the operating system
13.	speech_recognition	Library for performing speech recognition
14.	pyperclip	Clipboard operations (copy / paste text)
15.	webbrowser	Opens web pages in default browser
16.	subprocess	Spawns new processes and connects to their I/O
17.	datetime	Supplies classes for manipulating dates and times
18.	keyboard	Simulates keyboard events
19.	re	Used for regular expression operations
20.	urllib.error	Handles URL related exceptions
21.	socket	Used for handling network timeouts and errors
22.	time	Time related functions (e.g., sleep)
23.	pygetwindow	Gets and manipulates window titles and positions
24.	sentence_transformers	For loading SBERT models and sentence embedding
25.	torch	Deep learning library (Implicit via SBERT)

26.	absl	Abseil logging library for controlling log verbosity
-----	------	--

3.2.2 Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

Minimum Hardware Requirements

Table 4. Hardware Requirements

Hardware Requirement	Configuration
Processor	Intel Core i5 or higher
RAM	Minimum 8 GB
Microphone	For capturing user speech
Webcam	For capturing hand gestures
Storage	At least 5GB of free space for packages, models, and log files

3.3 NON-FUNCTIONAL REQUIREMENTS

- **Usability:** User-friendly web interface with clear buttons to start and stop each system.
- **Scalability:** Modular design allows independent upgrades to voice or gesture modules.
- **Maintainability:** Organized code with separate files for gesture control, voice assistant and web server logic.
- **Performance:** Real-time gesture and voice command processing with minimal latency.

- **Reliability:** Robust exception handling for speech and gesture modules to avoid crashes.
- **Portability:** Runs on any machine with Python and required hardware (microphone, webcam).
- **Security:** Runs locally with no external data sharing (beyond Google Speech API).

3.4 FLOWCHART OF OUR PROJECT

This flowchart illustrates the architecture of an adaptive accessibility system that integrates both voice and gesture-based controls for seamless laptop interaction. The system begins with a user interface that allows input via voice or webcam. For voice control, audio input is captured and processed through a speech recognition module, which converts it to text. The text is then semantically analyzed using Sentence BERT to map it to a valid system command. If the command is valid, it is executed; otherwise, an error message is displayed. Simultaneously, the gesture-based system uses video input to detect the user's hand using BlazePalm, followed by landmark analysis to capture hand movements. These gestures are recognized and mapped to specific operations like mouse control, volume, or brightness adjustments. Together, both modes offer an accessible and intuitive way for users to control their device.

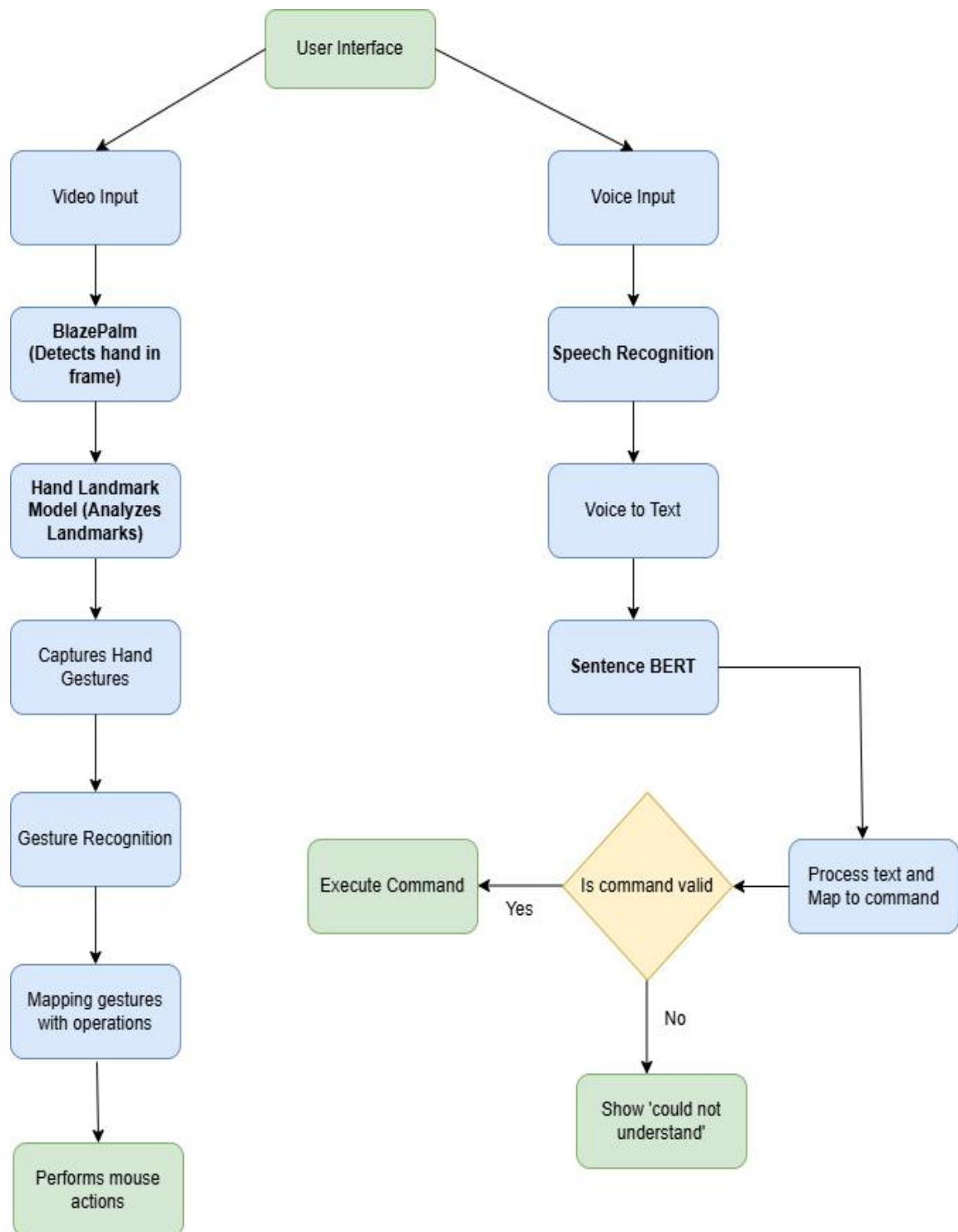


Fig. 3. System Architecture

3.5 ALGORITHMS

3.5.1 Sentence-BERT

Sentence-BERT (SBERT) is an advanced machine learning model built upon BERT (Bidirectional Encoder Representations from Transformers), specifically tailored for generating meaningful sentence embeddings. While BERT excels at token-level understanding, it lacks efficiency when it comes to comparing entire sentences. SBERT overcomes this by introducing a Siamese network architecture, enabling pairwise sentence comparisons through fixed-length vector embeddings.

In SBERT, each sentence is passed through a pre-trained BERT model. Instead of using the output of individual tokens directly, SBERT applies a pooling operation (such as mean pooling) over all token embeddings to produce a single dense vector for the entire sentence. This vector is a numerical representation of the sentence's meaning. Once embedded, two or more sentences can be compared using cosine similarity, a mathematical metric that determines how close the vectors (and hence, the meanings) are to each other.

The pooling operation computes the average of all token vectors in the sentence:

$$\text{Sentence Embedding} = \frac{1}{n} \sum_{i=1}^n \text{Token}_i$$

The cosine similarity formula used is:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Where:

- A and B are sentence embeddings,
- $\cdot$ is the dot product,
- $\|A\|$ is the Euclidean norm (or magnitude) of vector A.

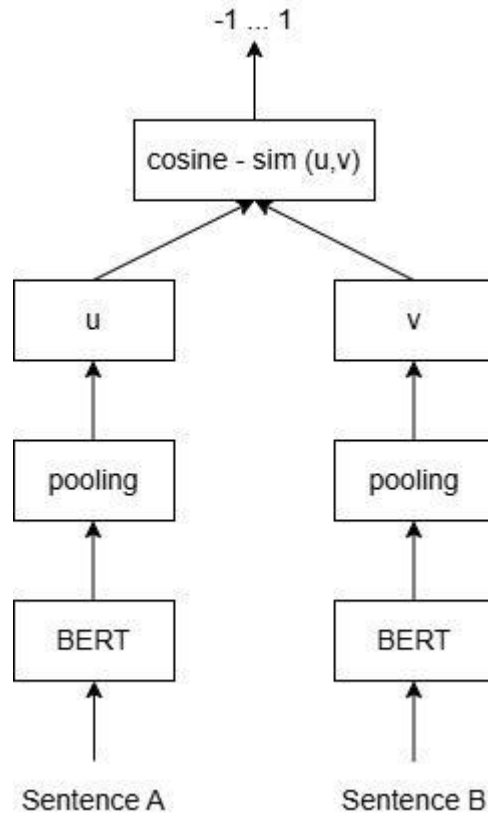


Fig. 4. Architecture of SBERT

3.5.2 BlazePalm

BlazePalm is a lightweight, real-time palm detection model developed by Google within the MediaPipe framework. Optimized for mobile and edge devices, it serves as the first stage in the MediaPipe Hands pipeline, detecting palms with high speed and accuracy. Unlike full-hand detectors, BlazePalm simplifies the task by focusing solely on the palm region, which is more stable and less deformable.

The model uses an anchor-based regression approach, predicting bounding boxes and object confidence scores relative to a set of predefined anchors. Non-Maximum Suppression (NMS) is applied to filter overlapping detections. Each output includes a rectangular bounding box and five key points: the palm center and four fingertip positions. These are then passed to the hand landmark model for detailed 3D keypoint estimation. While BlazePalm is a CNN trained end-to-end, bounding box dimensions are computed in post-processing using:

$$\text{width} = x_{\max} - x_{\min}, \quad \text{height} = y_{\max} - y_{\min}$$

This defines the region of interest for further processing.

By detecting only the palm initially, BlazePalm reduces computational load and improves robustness under occlusions, lighting changes, and cluttered background ideal for real-time gesture control applications.

3.5.3 Hand Landmark Model

Hand Landmark is the second stage in the MediaPipe Hands pipeline, responsible for precise localization of 3D keypoints on the detected hand. Given the palm region from the BlazePalm model, the hand landmark model predicts the spatial coordinates of 21 hand keypoints, including joints and fingertips. It is a regression-based convolutional neural network optimized for real-time performance on mobile and edge devices.

The model outputs 3D coordinates (x,y,z) for each keypoint, where x and y are normalized relative to the input image size, and z represents the depth (relative to the wrist keypoint, usually set as a reference point with z=0). These coordinates enable accurate tracking of hand poses and gestures. Although the model does not explicitly use geometric formulas during inference, post-processing often involves computing distances and angles for gesture recognition. For example, the Euclidean distance between two keypoints P1 (x1 ,y1 ,z1) and P2(x2,y2,z2) is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Such calculations are useful for detecting gestures like pinching, finger spread, or fist closure.

The hand landmark model also includes a handedness classifier that identifies whether the detected hand is left or right. By providing dense spatial information, the model enables accurate gesture recognition, sign language interpretation, and augmented reality applications all in real time with low latency and high reliability.

3.5.4 Speech Recognition Model

Speech Recognition is a powerful Python library designed for automatic speech recognition, enabling programs to convert spoken language into machine-readable text. It acts as a wrapper over several popular speech recognition engines, such as Google Web Speech API, CMU Sphinx, IBM Speech to Text, and others. In this project, it leverages the Google Web Speech API to perform transcription of live microphone input. Audio is captured using the Microphone class and processed through the Recognizer class, which handles noise adjustment, listening, and interaction with the backend API.

One notable feature is ambient noise calibration, which is handled by `recognizer.adjust_for_ambient_noise()`. This dynamically sets the energy threshold, improving recognition accuracy in noisy or unpredictable environments by filtering background sounds.

Although the library itself abstracts away the low-level mechanics, modern speech engines like Google's internally rely on deep learning-based acoustic and language models, often rooted in Bayesian inference. The probability of a word sequence W being spoken given an audio signal A is calculated using:

$$P(W|A) = \frac{P(A|W) \cdot P(W)}{P(A)}$$

Where:

- $P(A | W)$ represents the acoustic model (likelihood of the audio given a specific word sequence),
- $P(W)$ is the language model (prior probability of the word sequence),
- $P(W | A)$ is the posterior probability, i.e., the best match for transcription.

This probabilistic model allows the recognition system to select the most likely sentence or phrase from potentially ambiguous audio input.

4. DESIGN

4.1 INTRODUCTION

Unified Modeling Language (UML) is a standardized modeling language used to visually represent the design and structure of software systems. It provides a standard way to visualize how a system is designed. Good UML design is followed for a better and precise software output. The UML provides different constructs for specifying, visualizing, constructing and documenting the artifacts of software systems. UML diagrams are used to better understand the system, to maintain the document information about the system and emphasizes on the roles, actors, actions and process.

The key UML diagrams considered include:

- **Use-Case Diagram** – Represents the interaction between users (actors) and the system to define functional requirements.
- **Class Diagram** – Shows the static structure of the system, including classes, their attributes, methods, and relationships.
- **Sequence Diagram** – Describes how objects interact in a particular sequence of operations.
- **Activity Diagram** – Represents the flow of activities or processes within a system, often used for modeling workflows or business processes.

Relationships: Relationships define how different entities in UML diagrams interact with one another. The following types of relationships are commonly used:

- **Association** – Represents a relationship between two entities, describing how they interact.
- **Dependency** – Indicates that one entity is dependent on another, meaning any change in one affects the other.
- **Aggregation** – A special type of association that signifies a part-of relationship between objects.
- **Generalization** – Represents a parent-child relationship, where one entity is a specialized version of another.

- **Realization** – Defines a relationship in which one element specifies a behaviour or responsibility, and another element implements it.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

Use case diagrams capture the behaviour of a system and help to emphasize the requirements of the system. It describes the high-level functionalities of the system. It consists of the following artifacts,

- Actor
- Use case
- Secondary Actor and Use case Boundary

Actor

A role of a user that interacts with the system you're modeling is represented by an actor. The primary actor in this project is User. These users are again classified as depressed users and non-depressed users.

Use case

A use case is a function that a system does to help the user achieve their goal. There are 9 use cases which have their own functionality that can be useful in accomplishing the goals. The use cases send alert messages is a use case which includes detect and reply tweet use cases.

Relationships

Solid lines known as associations are used to show the relationships between actors and use cases. A connection signifies an actor's participation in a specific use case. Actors may also exhibit generalization relationships, which indicate specialization or inheritance.

System Boundary

It refers to the box that contains all of the system's use cases. It distinguishes the system under modeling from its surroundings and illustrates the extent of the system. Use case diagrams are useful tools for explaining a system's behaviour to stakeholders.

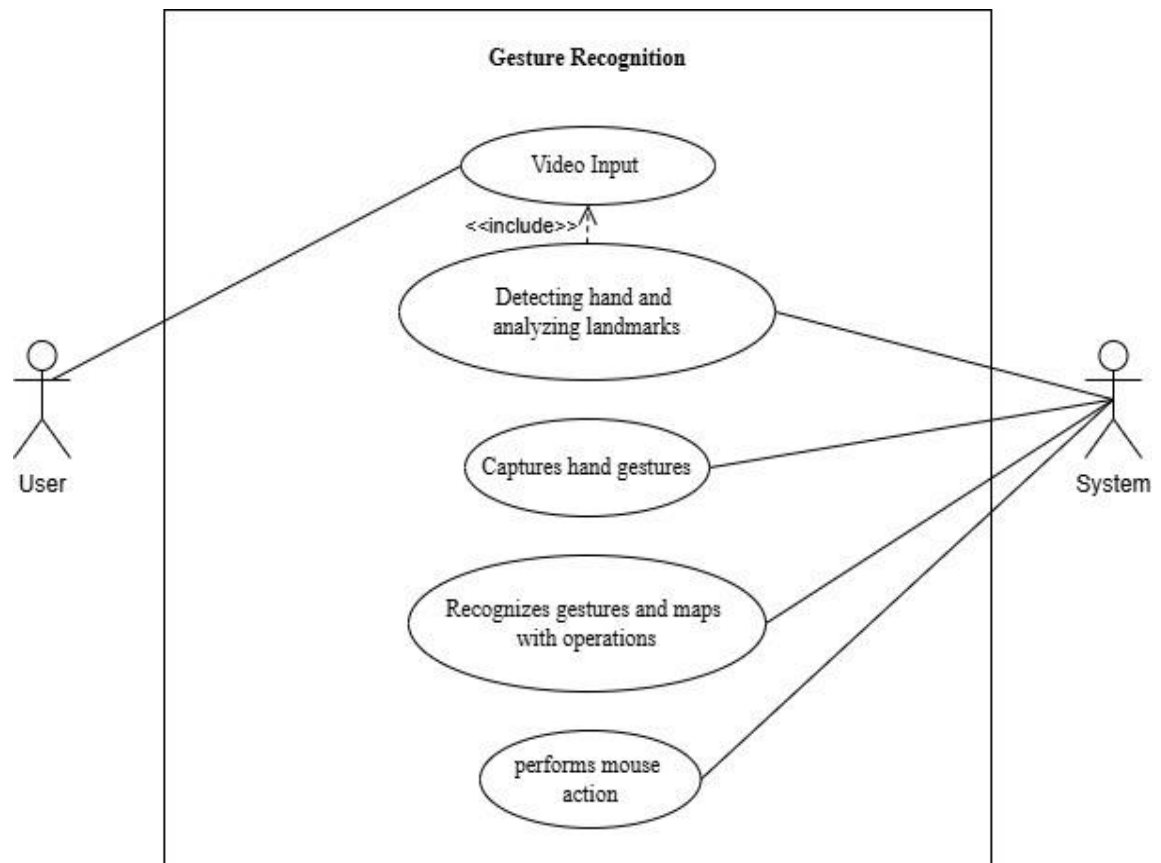


Fig. 5. Use Case Diagram 1 (Gesture Recognition)

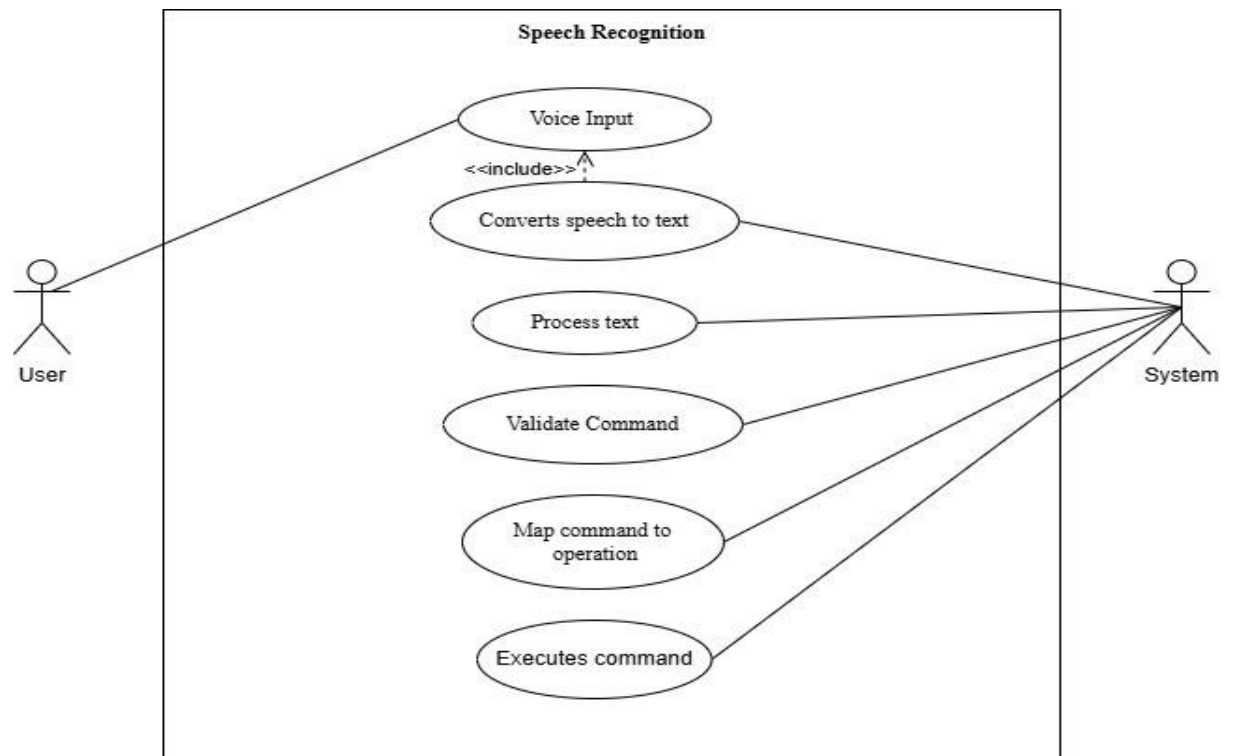


Fig. 6. Use Case Diagram 2 (Speech Recognition)

4.2.2 CLASS DIAGRAM

Class diagram is a static overview of the system used to highlight the important aspects as well as executables in the system. These are the only diagrams which can be mapped with the object-oriented systems. These diagrams show the responsibilities of a class and relationships among different classes in the system.

The class represents similar objects and consists of name, responsibilities and attributes. The classes considered in the project are Dataset, Data collection, Data Preprocessing, Data balancing, Model, and Classification. These are shown diagrammatically in the Fig. 8.

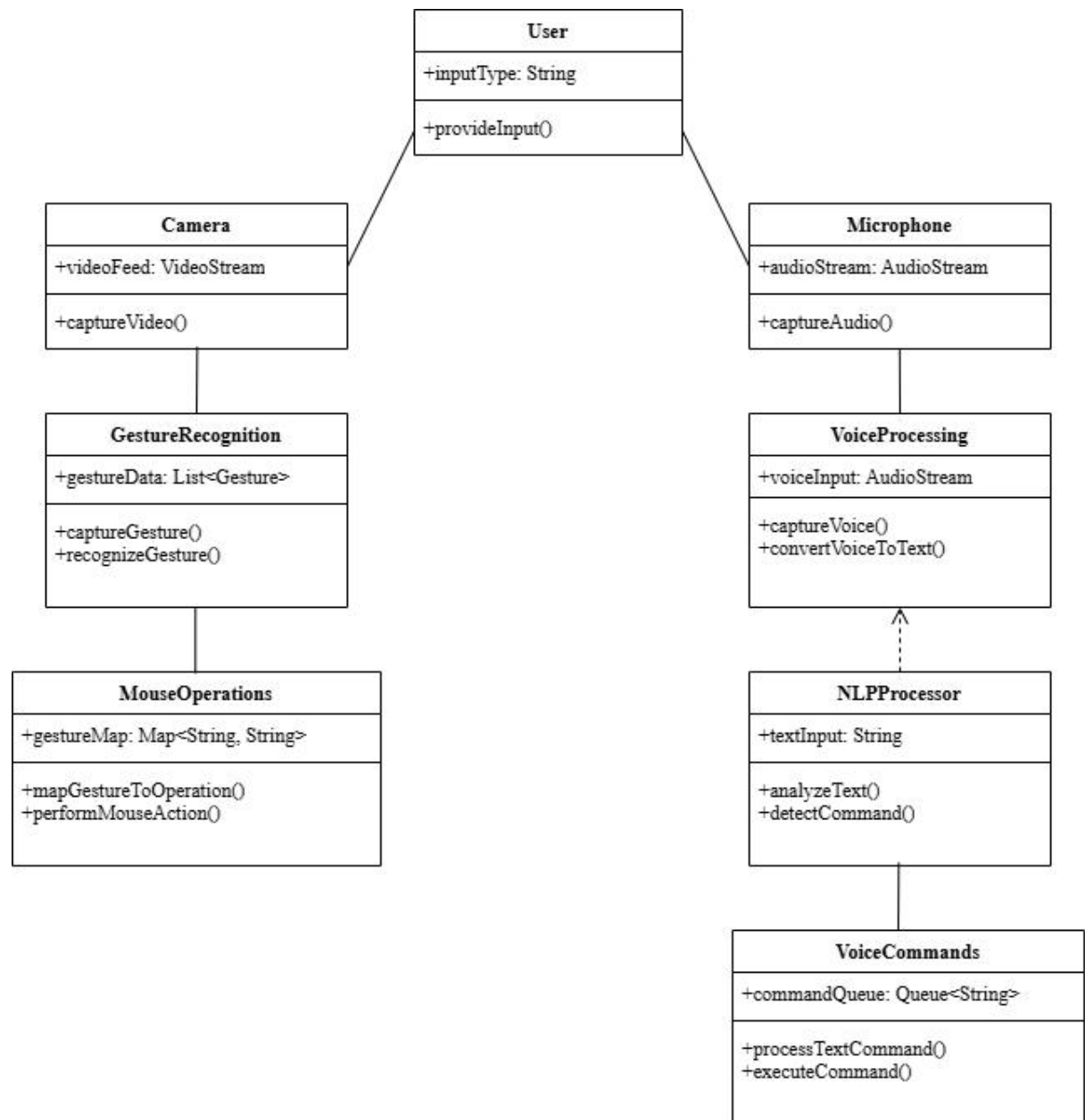


Fig. 7. Class Diagram

4.2.3 SEQUENCE DIAGRAM

The sequence diagram, also known as an event diagram, depicts the flow of messages through the system. It aids in the visualization of a variety of dynamic scenarios. It depicts communication between any two lifelines as a time-ordered series of events, as if these lifelines were present at the same moment. The message flow is represented by a vertical dotted line that extends across the bottom of the page in UML, whereas the lifeline is represented by a vertical bar. It encompasses both iterations and branching.

In Fig. 9, it shows how the lifeline works between user and its interface. It depicts how the data preprocessing and feature selection takes place. After that training the model and testing the model happens. After it gives the final result.

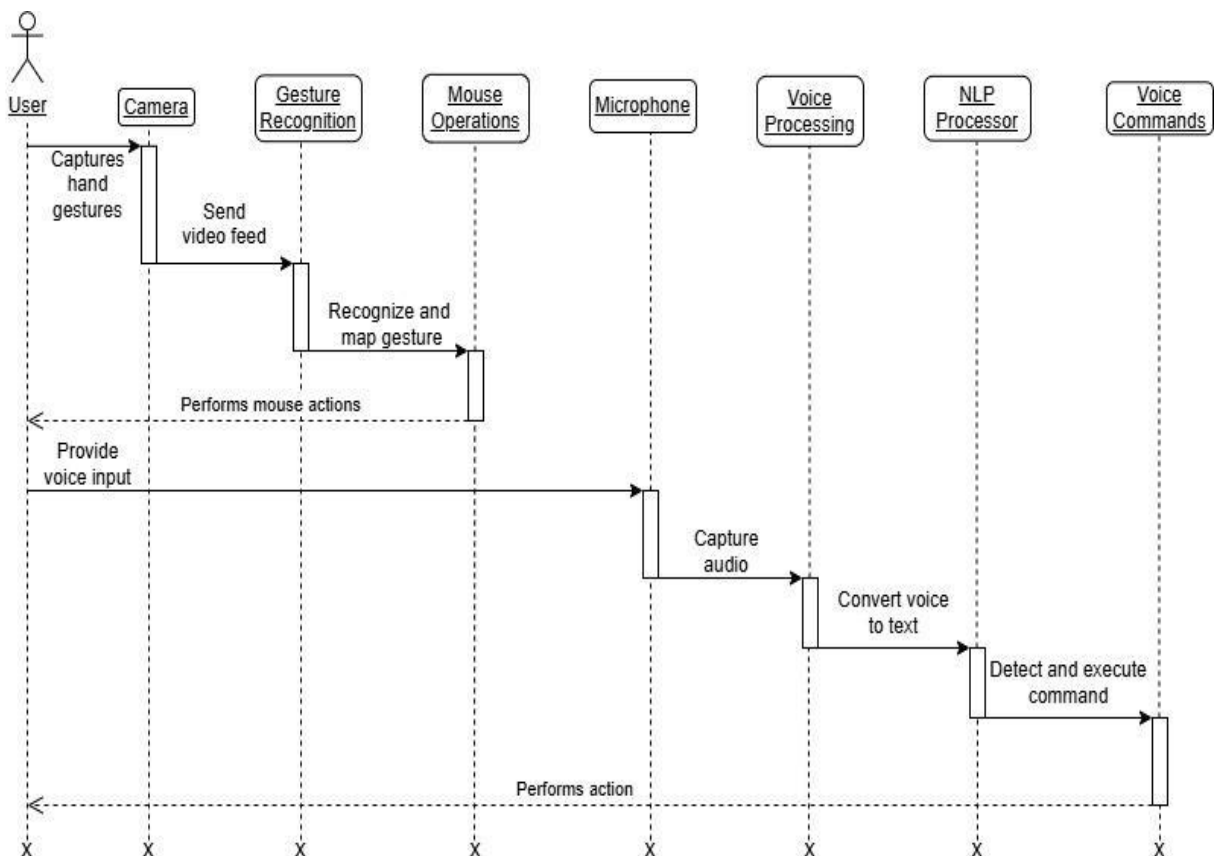


Fig. 8. Sequence Diagram

4.2.4 ACTIVITY DIAGRAM

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The basic purposes of activity diagrams are similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another. The only missing thing in the activity diagram is the message part.

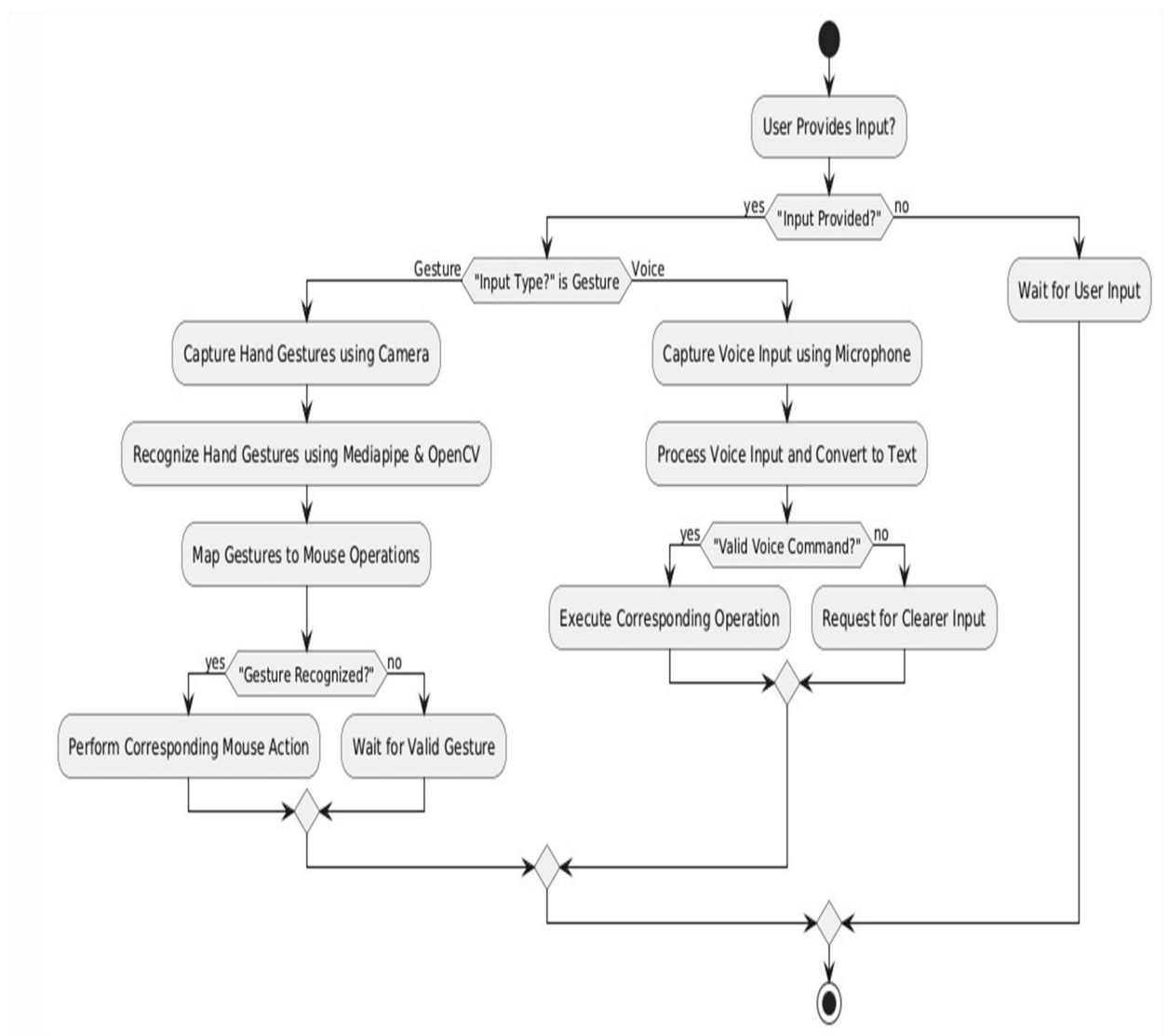


Fig. 9. Activity Diagram

4.3 MODULE DESIGN

4.3.1 System Module

In the development of the Adaptive Accessibility System for computer interaction, the project comprises two core input modalities: gesture-based control using hand landmarks and speech-based control for text and system command input. The system is designed to improve accessibility and user experience by enabling hands-free operation of computer functions. It uses webcam input for real-time hand gesture recognition and a microphone for capturing voice input. The system architecture is modular and integrates with a web interface that provides real-time logs of user activity and control feedback. Each module is explained below.

Data Collection

This module involves capturing real-time inputs from two sources, the system webcam and microphone. The webcam continuously records video frames containing hand movements, while the microphone listens for voice commands. No pre-stored datasets are used, as the system operates in real-time with dynamic user input. For testing and evaluation, a variety of hand gestures and spoken commands are performed under different lighting and environmental conditions to ensure robustness and adaptability of the system.

Data Preprocessing

For gesture recognition, each video frame is passed through a MediaPipe based hand tracking pipeline that detects and extracts 21 landmarks per detected hand. These landmarks are normalized and used for calculating finger positions, distances, and angles required for gesture recognition. To support both hands, a custom logic layer identifies left and right hand landmarks independently and applies gesture detection rules accordingly. In the case of voice input, raw audio is first converted to text using a speech recognition engine. The spoken sentence is then preprocessed by lowering case, removing filler words, and extracting core intent terms before feeding into the classifier for final interpretation.

Gesture and Speech-Based Input Processing

The system interprets various gestures to control mouse operations such as cursor movement, clicks, scrolling, and drag functions. For instance, a raised index finger controls cursor movement, while a pinching gesture triggers a click action. Other gestures like two-finger drag are mapped to scroll, and fist movement to volume or brightness adjustment. On the speech side, the system uses SBERT (Sentence-BERT) to generate sentence embeddings and compare them with a predefined command dataset. Commands are categorized as either typing input (for keyboard output) or control commands (such as opening applications or minimizing windows). Based on cosine similarity and thresholding, the system executes the closest matching command.

Model Training and Classification

While the hand gesture system relies on rule-based classification using landmark geometry, the voice command system uses a lightweight semantic similarity model for classification. A set of sample command phrases is embedded and stored in memory. Each user command is embedded in real-time and matched against this set using cosine similarity. No additional model training is needed, making the voice system lightweight and responsive. PyAutoGUI is used for executing both mouse and keyboard actions once the classification is complete.

System Integration and Visualization

The entire system is integrated into a single Python script and visualized through a Flask-based web application. This interface displays real-time log data for both gesture and voice actions. All outputs such as the recognized gesture, executed action, and spoken command are recorded in a shared `log_data` list. This log is rendered dynamically on the web page, giving users visual feedback of their interactions. This module also enables switching between gesture-only, voice-only, or combined modes, depending on user preference or accessibility needs.

Output Interpretation and Logging

Final output includes visual confirmation of the performed action (such as “Mouse clicked”, “Brightness increased”, or “Typed: Hello”) along with time stamped logs. For gestures, a real-time visual of the hand with detected landmarks may be optionally displayed. All actions are stored in a log file or database for further analysis or reporting. This allows the system to be evaluated for effectiveness, latency, and user experience over time.

4.4 CONCLUSION

The design of the adaptive accessibility system is modular and efficient, ensuring that each component from gesture tracking and voice recognition to action execution and web-based logging works cohesively to deliver a smooth user experience. Each module is responsible for a specific functionality, contributing to the overall performance and usability of the system. Real-time video and audio inputs are effectively processed through optimized pipelines, enabling accurate and responsive control without external devices. The separation of modules allows easy debugging, scalability, and the potential for future enhancements such as multi-language voice support or gesture training. This well-structured module design ensures that the system is not only functional but also flexible and accessible for a wide range of users.

5. IMPLEMENTATION AND RESULTS

5.1 INTRODUCTION

The project's implementation stage is when the theoretical design is translated into a workable system. As a result, it can be seen as the most crucial stage in ensuring the success of a new system and giving the user confidence that the system will work and be effective. The implementation step entails meticulous planning, research of the existing system and its implementation limitations, designing of changeover methods, and evaluation of changeover methods.

5.2 SOFTWARE DESIGN

5.2.1 Front End Design

HTML

Hypertext markup language (HTML) is used for texts intended to be viewed in a web browser. Technologies such as Cascading Style Sheets (CSS) and programming languages like JavaScript can help. Web browsers receive HTML documents from a web server or locally stored files and convert them to multimedia web pages. HTML originally featured cues for the document's look and described the structure of a web page logically. HTML tags are the components that make up HTML pages. Images and other objects, such as interactive forms, can be embedded in the produced page using HTML structures.

HTML allows you to create organized documents by indicating structural semantics for text elements like headers, paragraphs, lists, links, quotations, and other elements. Tags, which are written in angle brackets, separate HTML elements. Tags like `img />` and `input />` insert content into the page immediately. Other tags, such as `p>`, surround and offer information about document text and may comprise sub elements of other tags. Browsers do not show HTML tags; instead, they use them to interpret the page's content.

CSS

CSS, or Cascading Style Sheets, is a simple design language intended to make the process of making web pages presentable easier. Styles can be applied to web pages using CSS. More crucially, CSS allows you to do so without relying on the HTML that each web page contains. CSS is simple to learn and understand, but it gives you a lot of power over how an HTML document looks. Cascading Style Sheets (CSS) is a key web development tool that defines

the appearance and layout of HTML texts. It allows web designers and developers to manage several aspects of a webpage's appearance, including fonts, colors, spacing, and position. CSS operates by choosing HTML components and applying styles to them according to rules stated in style sheets. These style sheets can be incorporated in HTML texts, included externally, or used dynamically with scripting languages. CSS also provides cascading, inheritance, and specificity, which allows for efficient and structured styling across web pages while maintaining design consistency and flexibility.

5.2.2 Backend Design

Flask

Flask is a lightweight and flexible web framework for Python that allows developers to build web applications quickly and with minimal overhead. It is designed with simplicity and ease of use in mind, making it an excellent choice for both beginners and experienced developers. Flask provides essential features such as routing, request handling, and template rendering, but leaves many other functionalities to be handled by extensions, giving developers the freedom to add only what they need. This "micro" framework approach makes Flask highly extensible and ideal for creating APIs, single-page applications, or small web apps. With its minimalistic structure and ease of integration, Flask is widely used for building RESTful services and data-driven applications.

Data Collection

Instead of relying on pre-collected datasets, our project captures real-time data through live inputs. The system leverages live video feeds for hand gesture recognition and real-time audio input for voice commands. The hand gesture model continuously processes video frames from a webcam, detecting various hand gestures such as pointing, clicking, scrolling, and others. For the voice-based command system, the system listens to live audio input using a microphone to recognize spoken commands and perform corresponding actions on the system.

Data Preprocessing

As the inputs are live, preprocessing is performed in real time to ensure efficient and accurate interpretation:

- **Hand Gesture Preprocessing:**

Each video frame is captured, resized, and normalized. MediaPipe is used to extract hand landmarks, which are then used to interpret gestures like click, scroll, or move. These operations help reduce background noise and focus only on relevant features.

- **Voice Command Preprocessing:**

Audio input is continuously streamed and converted into text using speech-to-text models. This involves processing the raw audio signal into a format suitable for recognition.

Model Integration and Execution

The gesture and voice recognition modules are integrated into a unified backend pipeline using the Flask framework, enabling centralized control through a single interface. Though developed in separate Python scripts, Flask coordinates their execution, ensuring seamless communication.

The system is capable of handling simultaneous gesture and voice inputs, allowing both modes to operate in parallel without interruption. This enhances responsiveness and enables flexible interaction based on user preference.

A shared logging mechanism is used to capture and store outputs from both modules. These logs are dynamically displayed on a web-based dashboard in real time, providing users with continuous feedback. This design improves system usability, transparency, and user experience.

5.3 IMPLEMENTATION

Front-end

app.py

```
from flask import Flask, render_template, jsonify, request, Response
import subprocess
import threading
import time
import os

app = Flask(__name__)
gesture_process = None
speech_process = None

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/about")
def about():
    return render_template("about.html")

@app.route("/voice")
def voice():
    return render_template("voice.html")

@app.route("/hand")
def hand():
    return render_template("hand.html")

@app.route("/howtouse")
def howtouse():
    return render_template("howtouse.html")

@app.route("/start_gesture", methods=["POST"])
def start_gesture():
    global gesture_process
    if gesture_process is None:
        gesture_process = subprocess.Popen(["python", "gesture_controller.py"])
        return jsonify({"status": "Gesture Controller started"})
    return jsonify({"status": "Gesture Controller is already running"})
```

```

@app.route("/stop_gesture", methods=["POST"])
def stop_gesture():
    global gesture_process
    if gesture_process:
        gesture_process.terminate()
        gesture_process = None
    return jsonify({"status": "Gesture Controller stopped"})
    return jsonify({"status": "Gesture Controller is not running"})

@app.route("/start_speech", methods=["POST"])
def start_speech():
    global speech_process
    if speech_process is None:
        # Open a log file to capture the output
        log_file = open("speech_output.log", "a")
        # Start the speech commander process and capture stdout
        speech_process = subprocess.Popen(
            ["python", "speech_commander.py"],
            stdout=log_file, # Capture stdout to a log file
            stderr=subprocess.STDOUT
        )
        # Log that the Speech Commander has started
        with open("speech_output.log", "a") as log:
            log.write("Speech Commander started\n")
        return jsonify({"status": "Speech Commander started"})
    return jsonify({"status": "Speech Commander is already running"})

@app.route("/stop_speech", methods=["POST"])
def stop_speech():
    global speech_process
    if speech_process:
        speech_process.terminate()
        # Log that the Speech Commander has stopped
        with open("speech_output.log", "a") as log:
            log.write("Speech Commander stopped\n")
        speech_process = None

```

```

        return jsonify({"status": "Speech Commander stopped"})
    return jsonify({"status": "Speech Commander is not running"})
@app.route("/speech_logs")
def stream_logs():
    def generate():
        logfile_path = "speech_output.log"
        # Open the log file to read the contents
        with open(logfile_path, "r") as f:
            f.seek(0, os.SEEK_END) # Move to end of file
            # Continuously read new lines from the log file
            while True:
                line = f.readline()
                if line:
                    yield f"data: {line.strip()}\n\n" # Send the log line as SSE
                else:
                    time.sleep(0.5) # Wait before checking for new lines
    return Response(generate(), mimetype="text/event-stream")
@app.route("/clear_logs", methods=["POST"])
def clear_logs():
    try:
        with open("speech_output.log", "w") as log_file:
            log_file.truncate(0)
        return jsonify({"status": "Log cleared"})
    except Exception as e:
        return jsonify({"status": f"Error clearing log: {str(e)}"})
if __name__ == "__main__":
    app.run(debug=True, threaded=True)

```

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="icon" href="{{ url_for('static', filename='favicon.png') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"/>
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"></script>
    <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
  </head>
  <body>
    <div id="SectionIndex">
      <nav class="navbar navbar-expand-lg navbar-light">
        <div class="container">
          <a class="navbar-brand d-flex align-items-center" href="#">
            
            <span class="h4 mb-0 navbar-heading">Adaptive Accessibility System</span>
          </a>
          <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNavAltMarkup">
            <span class="navbar-toggler-icon"></span>
          </button>
          <div class="collapse navbar-collapse" id="navbarNavAltMarkup">
            <div class="navbar-nav ml-auto">
              <a class="nav-link active navbar" href="{{ url_for('about') }}"><u>About</u></a>
              <a class="nav-link active navbar" href="{{ url_for('howtouse') }}"><u>How to use</u></a>
            </div>
          </div>
        </div>
      </div>
    </div>
```



```

</nav>
<div class="banner-section-bg-container d-flex justify-content-center flex-column">
  <div class="text-center container">
    <h1 class="heading">Virtual Mouse and Keyboard</h1>
    <p>Control your computer with gestures and voice commands.</p>
    <!-- Gesture Controls -->
    <div class="controls">
      <h2 class="control-heading">Gesture Controller</h2>
      <button id="start-gesture" class="control-button btn btn-success">Start</button>
      <button id="stop-gesture" class="control-button btn btn-danger">Stop</button>
      <p id="gesture-status">Status: Inactive</p>
    </div>
    <!-- Speech Controls -->
    <div class="controls">
      <h2 class="control-heading">Speech Commander</h2>
      <button id="start-speech" class="control-button btn btn-success">Start</button>
      <button id="stop-speech" class="control-button btn btn-danger">Stop</button>
      <p id="speech-status">Status: Inactive</p>
    </div>
    <!-- Log Section -->
    <div class="log mb-3">
      <h2 class="Log1">Log</h2>
      <div id="log-output" class="border p-2 rounded bg-light" style="height: 200px;
overflow-y: auto;"></div>
    </div>
    <!-- Clear Log Button -->
    <div class="text-center mb-5">
      <button id="clear-log" class="btn btn-dark">Clear Log</button>
    </div>
  </div>
</div>
<script>
  let logEventSource = null;

```

```

function updateLog(message) {
  const logOutput = document.getElementById("log-output");
  logOutput.innerHTML += `<p>${message}</p>`;
  logOutput.scrollTop = logOutput.scrollHeight;
}

function startLogStream() {
  if (logEventSource) {
    logEventSource.close();
  }
  logEventSource = new EventSource('/speech_logs');
  logEventSource.onmessage = function(event) {
    updateLog(event.data);
  };
}

async function handleButtonClick(endpoint, statusElement) {
  const response = await fetch(endpoint, { method: "POST" });
  const data = await response.json();
  updateLog(data.status);
  statusElement.textContent = `Status: ${data.status.includes("started") ? "Active" :
"Inactive"}`;
}

document.getElementById("start-gesture").addEventListener("click", () => {
  handleButtonClick("/start_gesture", document.getElementById("gesture-status"));
});

document.getElementById("stop-gesture").addEventListener("click", () => {
  handleButtonClick("/stop_gesture", document.getElementById("gesture-status"));
});

document.getElementById("start-speech").addEventListener("click", async () => {
  await handleButtonClick("/start_speech", document.getElementById("speech-status"));
  startLogStream();
});

document.getElementById("stop-speech").addEventListener("click", () => {
  handleButtonClick("/stop_speech", document.getElementById("speech-status"));
  if (logEventSource) {

```

```

        logEventSource.close();
        logEventSource = null;
    }
});
document.getElementById("clear-log").addEventListener("click", async () => {
    const response = await fetch("/clear_logs", { method: "POST" });
    const data = await response.json();
    updateLog(data.status);
    document.getElementById("log-output").innerHTML = "";
});
</script>
</body>
</html>

```

Integration Model

gesture_controller.py

```

import cv2
import mediapipe as mp
import pyautogui
import math
from enum import IntEnum
from google.protobuf.json_format import MessageToDict
# Gesture Encoding
class Gest(IntEnum):
    FIST = 0
    INDEX = 8
    MID = 4
    PINCH_MINOR = 36
    PINCH_MAJOR = 35
    PALM = 31
# Hand Label (Right or Left)
class HLabel(IntEnum):
    MINOR = 0
    MAJOR = 1

```

```

# Hand Recognition
class HandRecog:
    def __init__(self, hand_label):
        self.hand_label = hand_label
        self.finger = 0
        self.ori_gesture = Gest.PALM
        self.prev_gesture = Gest.PALM
    def update_hand_result(self, hand_result):
        self.hand_result = hand_result
    def get_dist(self, point):
        dist = (self.hand_result.landmark[point[0]].x - self.hand_result.landmark[point[1]].x) ** 2
        dist += (self.hand_result.landmark[point[0]].y - self.hand_result.landmark[point[1]].y)
        **2
        return math.sqrt(dist)
    def set_finger_state(self):
        if self.hand_result is None:
            return
        # Finger states (Open or Closed)
        points = [[8, 5, 0], [12, 9, 0], [16, 13, 0], [20, 17, 0]]
        self.finger = 0
        for idx, point in enumerate(points):
            dist = self.get_dist(point[:2])
            dist2 = self.get_dist(point[1:])
            ratio = round(dist / dist2, 1) if dist2 != 0 else 0.1
            self.finger = self.finger << 1
            if ratio > 0.5:
                self.finger = self.finger | 1
    def get_gesture(self):
        current_gesture = Gest.PALM
        if self.finger == Gest.PINCH_MINOR:
            current_gesture = Gest.PINCH_MINOR
        elif self.finger == Gest.PINCH_MAJOR:
            current_gesture = Gest.PINCH_MAJOR
        elif self.finger == Gest.FIST:

```

```

        current_gesture = Gest.FIST
    elif self.finger == Gest.INDEX:
        current_gesture = Gest.INDEX
    self.prev_gesture = current_gesture
    return current_gesture

# Controller for Gesture Action Handling
class Controller:
    @staticmethod
    def get_position(hand_result):
        point = 9
        position = [hand_result.landmark[point].x, hand_result.landmark[point].y]
        sx, sy = pyautogui.size()
        x = int(position[0] * sx)
        y = int(position[1] * sy)
        return x, y

    @staticmethod
    def handle_controls(gesture, hand_result):
        x, y = Controller.get_position(hand_result)
        # Simulate mouse controls based on detected gestures
        if gesture == Gest.FIST:
            pyautogui.mouseDown(button="left")
            pyautogui.moveTo(x, y, duration=0.1)
        elif gesture == Gest.INDEX:
            pyautogui.rightClick(x, y)
        elif gesture == Gest.PINCH_MINOR:
            pyautogui.scroll(120 if Controller.pinchlv > 0 else -120)

# Gesture Controller for Real-time Gesture Processing
class GestureController:
    def __init__(self):
        self.cap = cv2.VideoCapture(0)
        self.hr_major = None
        self.hr_minor = None
    def classify_hands(self, results):
        left, right = None, None

```

```

try:
    handedness_dict = MessageToDict(results.multi_handedness[0])
    if handedness_dict['classification'][0]['label'] == 'Right':
        right = results.multi_hand_landmarks[0]
    else:
        left = results.multi_hand_landmarks[0]
except:
    pass

try:
    handedness_dict = MessageToDict(results.multi_handedness[1])
    if handedness_dict['classification'][0]['label'] == 'Right':
        right = results.multi_hand_landmarks[1]
    else:
        left = results.multi_hand_landmarks[1]
except:
    pass

self.hr_major = right if right else left
self.hr_minor = left if left else right

def start(self):
    handmajor = HandRecog(HLabel.MAJOR)
    handminor = HandRecog(HLabel.MINOR)
    with mp.solutions.hands.Hands(max_num_hands=2, min_detection_confidence=0.5,
min_tracking_confidence=0.5) as hands:
        while self.cap.isOpened():
            success, image = self.cap.read()
            if not success:
                print("Ignoring empty camera frame.")
                continue
            image = cv2.cvtColor(cv2.flip(image, 1), cv2.COLOR_BGR2RGB)
            image.flags.writeable = False
            results = hands.process(image)
            image.flags.writeable = True
            image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
            if results.multi_hand_landmarks:

```

```

        self.classify_hands(results)
        handmajor.update_hand_result(self.hr_major)
        handminor.update_hand_result(self.hr_minor)
        handmajor.set_finger_state()
        handminor.set_finger_state()
        gest_name = handminor.get_gesture()
        if gest_name != Gest.PALM:
            Controller.handle_controls(gest_name, handminor.hand_result)
        gest_name = handmajor.get_gesture()
        if gest_name != Gest.PALM:
            Controller.handle_controls(gest_name, handmajor.hand_result)
        # Draw hand landmarks for feedback
        for hand_landmarks in results.multi_hand_landmarks:
            mp.solutions.drawing_utils.draw_landmarks(image, hand_landmarks,
                mp.solutions.hands.HAND_CONNECTIONS)
        cv2.imshow('Gesture Controller', image)
        if cv2.waitKey(5) & 0xFF == 13:
            break
    self.cap.release()
    cv2.destroyAllWindows()

# Initialize and start the Gesture Controller
gc1 = GestureController()
gc1.start()

```

speech_commander.py

```

import sys
import os

# Force stdout to be line-buffered
sys.stdout = os.fdopen(sys.stdout.fileno(), 'w', buffering=1)

# Suppress specific warnings
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
os.environ["TRANSFORMERS_VERBOSITY"] = "error"

import speech_recognition as sr
import pyautogui

```

```

import pyperclip
import webbrowser
import subprocess
import datetime
import keyboard
import re
import urllib.error
import socket
import time
import pygetwindow as gw
import screen_brightness_control as sbc
from sentence_transformers import SentenceTransformer, util
# === Load SBERT model ===
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
# === Commands & Embeddings ===
command_phrases = [
    "increase volume", "decrease volume", "increase brightness", "decrease brightness",
    "open notepad", "close notepad", "save file", "take screenshot",
    "press windows button", "escape", "search", "start typing",
    "stop typing", "activate typing mode", "activate command mode",
    "exit the assistant", "create new line", "backspace", "select everything",
    "copy this", "cut this", "paste here", "press tab", "arrow up", "arrow down",
    "arrow left", "arrow right", "start math mode", "stop math mode", "caps lock",
    "space", "delete word", "delete line", "open youtube", "open google",
    "open gmail", "open github", "open chat gpt", "press enter", "press end",
    "open start menu", "pause assistant", "stop listening",
    "resume assistant", "continue listening"
]
command_labels = [
    "volume_up", "volume_down", "brightness_up", "brightness_down",
    "open_notepad", "close_notepad", "save_file", "screenshot",
    "windows_button", "escape_button", "search", "start_typing",
    "stop_typing", "activate_typing", "activate_commands",
    "exit", "new_line", "backspace", "select_all",

```



```

"copy", "cut", "paste", "tab", "up", "down", "left", "right",
"start_math", "stop_math", "caps_lock",
"space", "delete_word", "delete_line", "open_youtube", "open_google",
"open_gmail", "open_github", "open_chatgpt", "press_enter", "press_end",
"start_menu", "pause_assistant", "pause_assistant",
"resume_assistant", "resume_assistant"
]
command_embeddings = model.encode(command_phrases, convert_to_tensor=True)
# === Assistant state ===
mode = 'command'
typing_active = False
start_typing = False
math_mode = False
caps_lock_on = False
paused = False
CONFIDENCE_THRESHOLD = 0.4
# === Logging ===
LOG_FILE = "command_log.txt"
if not os.path.exists(LOG_FILE):
    with open(LOG_FILE, "w") as f:
        f.write("Timestamp\tConfidence\tUser Command\n")
def log_command(user_input, confidence):
    now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FILE, "a") as f:
        f.write(f"{now}\t{confidence:.2f}\t{user_input}\n")
# === Speech Recognition ===
def recognize_speech():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        #print("Calibrating for background noise...")
        recognizer.adjust_for_ambient_noise(source, duration=1)
        #print(f"Adjusted energy threshold: {recognizer.energy_threshold}")
        print(f"\nListening in {mode} mode...")
        try:

```

```

        audio = recognizer.listen(source, timeout=8)
        user_input = recognizer.recognize_google(audio).lower()
        print(f"You said: {user_input}")
        return user_input
    except sr.UnknownValueError:
        print("Could not understand.")
    except sr.RequestError:
        print("Speech recognition service error.")
    except sr.WaitTimeoutError:
        print("Timeout. Try again.")
    except urllib.error.URLError:
        print("Network error occurred.")
    except sr.TimeoutError:
        print("Connection timed out.")
    except socket.timeout:
        print("Socket timeout occurred.")
    except sr.Exception:
        print("Unexpected error.")
    return None

# === Predict Intent ===
def predict_intent(user_input):
    user_embedding = model.encode(user_input, convert_to_tensor=True)
    cosine_scores = util.cos_sim(user_embedding, command_embeddings)[0]
    best_match_idx = cosine_scores.argmax().item()
    best_match_score = cosine_scores[best_match_idx].item()
    log_command(user_input, best_match_score)
    if best_match_score >= CONFIDENCE_THRESHOLD:
        return command_labels[best_match_idx]
    else:
        return None

# === Format Math Text ===
def format_math_equation(text):
    text = text.lower()
    text = text.replace("pai", "pi").replace("route of", "root of").replace("root off", "root of")

```

```

replacements = {
    # Operators and comparisons
    "plus": "+", "minus": "-", "times": "", "multiplied by": "",
    "divided by": "/", "mod": "%", "equals": "=", "equal to": "=",
    "double equal": "==", "not equal": "≠",
    # Logical
    "greater than or equal to": "≥", "less than or equal to": "≤",
    "greater than": ">", "less than": "<",
    # Powers
    "to the power of": "^", "square": "^2", "cube": "^3", "power": "^",
    # Numbers
    "hundred": "00", "thousand": "000", "million": "000000",
    "zero": "0", "one": "1", "two": "2", "three": "3", "four": "4",
    "five": "5", "six": "6", "seven": "7", "eight": "8", "nine": "9",
    "ten": "10",
    # Math symbols
    "degree": "°", "pi": "π", "angle": "θ", "alpha": "α", "beta": "β",
    "gamma": "γ", "gama": "γ", "delta": "δ", "micro": "μ", "infinity": "∞",
    "eta": "η", "neeta": "η", "point": ".",
    # Roots
    "square root of": "√", "cube root of": "∛", "nth root of": "ⁿ√", "root of": "√",
    # Grouping
    "open parenthesis": "(", "close parenthesis": ")",
    # Variables
    "ex": "x", "why": "y", "zed": "z", "zee": "z",
    # Calculus and summation
    "summation": "Σ", "integral": "∫"}
}

# Sort keys longest-first to prevent partial replacements
sorted_keys = sorted(replacements.keys(), key=len, reverse=True)
for key in sorted_keys:
    if key in text:
        text = text.replace(key, replacements[key])
# Handle fractions like "3 by 4" → "3/4"

```

```

text = re.sub(r'(\d+)\s+by\s+(\d+)', r'\1/\2', text)
# Merge things like "1 000000" into "1000000"
text = re.sub(r'(\d)\s+([0])\b', r'\1\2', text)
# Remove spaces around math operators (except between symbols)
text = re.sub(r's*([\+ \- \= ^ % ° \sqrt[3]{ } \neq \< \>])\s', r'\1', text)
# Add single space between words/symbols (if not connected operators)
text = re.sub(r'([\^ \s])([\alpha-\omega \Omega a-zA-Z \pi \theta \gamma \alpha \beta \delta \infty \sum])', r'\1 \2', text)
# Allow only valid characters
allowed_chars = set("0123456789+-*/=^%°\pi\theta\gamma\alpha\beta\delta\eta\mu\sqrt[3]{ }^\infty\gtrless\lessgtr().xyz#\sum")
output = ".join(c for c in text if c in allowed_chars)
return (output.strip() + " ") if output.strip() else ""

# === Execute Commands ===
def execute_command(user_input):
    global mode, typing_active, start_typing, math_mode, caps_lock_on
    if not user_input:
        return True
    intent = predict_intent(user_input)
    if typing_active and not start_typing:
        allowed_commands = ["start_typing", "stop_typing", "activate_commands"]
        if intent not in allowed_commands:
            print("Invalid command in typing mode. Please say 'start typing', 'stop typing', or
'activate command mode")
            return True
    if typing_active and start_typing:
        if intent == "start_math":
            math_mode = True
            print("Math mode enabled.")
            return True
        elif intent == "stop_math":
            math_mode = False
            print("Math mode disabled.")
            return True
        elif intent == "stop_typing":
            start_typing = False

```

```

    print("Typing stopped.")
    return True
elif intent == "exit":
    if math_mode:
        math_mode = False
        print("Math mode disabled.")
    else:
        start_typing = False
        print("Exiting typing mode. Back to command mode.")
        mode = "command"
        typing_active = False
    return True
else:
    if math_mode:
        output_text = format_math_equation(user_input)
        if output_text:
            pyperclip.copy(output_text)
            pyautogui.hotkey("ctrl", "v")
            print(f"Typed (math): {output_text}")
        else:
            print("No valid math symbols found in input.")
    else:
        if caps_lock_on:
            user_input = user_input.upper()
            keyboard.write(user_input.strip() + " ")
            print(f"Typed: {user_input}")
        return True
if intent is None:
    if not typing_active:
        print("I'm not sure what you meant. Could you rephrase?")
    return True
if intent == "caps_lock":
    caps_lock_on = not caps_lock_on
    pyautogui.press("capslock")

```

```

    state = "ON" if caps_lock_on else "OFF"
    print(f"Caps Lock toggled: {state}")
    return True
if intent == "activate_typing":
    mode = "typing"
    typing_active = True
    start_typing = False
    math_mode = False
    print("Typing mode activated.")
    return True
if intent == "start_typing" and typing_active:
    start_typing = True
    print("Start speaking to type...")
    return True
if intent == "stop_typing":
    start_typing = False
    print("Typing paused.")
    return True
if intent == "activate_commands":
    mode = "command"
    typing_active = False
    start_typing = False
    math_mode = False
    print("Command mode activated.")
    return True
if intent == "exit":
    print("Exiting voice assistant.")
    return False
if intent == "search":
    query = user_input.replace("search", "").strip()
    if query:
        webbrowser.open(f"https://www.google.com/search?q={query}")
        print(f"Searching Google for: {query}")
    else:

```

```

        print("What do you want to search?")
    return True
if intent == "open_google":
    webbrowser.open("https://www.google.com")
    print("Opening Google...")
    return True
if intent == "open_youtube":
    webbrowser.open("https://www.youtube.com")
    print("Opening YouTube...")
    return True
if intent == "open_gmail":
    webbrowser.open("https://mail.google.com")
    print("Opening Gmail...")
    return True
if intent == "open_github":
    webbrowser.open("https://github.com")
    print("Opening GitHub...")
    return True
if intent == "open_chatgpt":
    webbrowser.open("https://chat.openai.com")
    print("Opening ChatGPT...")
    return True
if intent == "open_notepad":
    subprocess.Popen("notepad.exe")
    print("Notepad opened.")
    return True
if intent == "close_notepad":
    pyautogui.hotkey("alt", "f4")
    pyautogui.press("right")
    pyautogui.press("enter")
    print("Notepad closed.")
    return True
if intent == "save_file":
    pyautogui.hotkey("ctrl", "s")

```

```

print("Say filename:")
filename = recognize_speech()
if filename:
    pyautogui.write(filename + ".txt")
    pyautogui.press("enter")
    print(f"File saved as {filename}.txt")
    return True
if intent == "volume_up":
    pyautogui.press("volumeup", presses=5)
    print("Volume increased.")
    return True
if intent == "volume_down":
    pyautogui.press("volumedown", presses=5)
    print("Volume decreased.")
    return True
if intent == "brightness_up":
    try:
        b = sbc.get_brightness(display=0)[0]
        sbc.set_brightness(min(b + 10, 100), display=0)
        print("Brightness increased.")
    except Exception as e:
        print(f"Brightness error: {e}")
    return True
if intent == "brightness_down":
    try:
        b = sbc.get_brightness(display=0)[0]
        sbc.set_brightness(max(b - 10, 0), display=0)
        print("Brightness decreased.")
    except Exception as e:
        print(f"Brightness error: {e}")
    return True
if intent == "screenshot":
    screenshot = pyautogui.screenshot()
    print("Say 'accept' to save or 'reject' to discard:")

```



```

response = recognize_speech()
if response == "accept":
    print("Say screenshot name:")
    name = recognize_speech()
    if name:
        screenshot.save(f'{name}.png')
        print(f'Screenshot saved as {name}.png')
    else:
        print("Screenshot discarded.")
    return True
if intent == "windows_button":
    pyautogui.press("win")
    print("Windows key pressed.")
    return True
if intent == "escape_button":
    pyautogui.press("esc")
    print("Escape key pressed.")
    return True
if intent == "pause_assistant":
    paused = True
    print("Assistant paused.")
    return True
if intent == "resume_assistant":
    paused = False
    print("Assistant resumed.")
    return True
if intent == "start_menu":
    while True:
        pyautogui.press("win")
        print("Say what you want to search in the Start menu:")
        search_query = recognize_speech()
        if not search_query:
            print("Didn't catch that. Exiting Start menu search.")
            pyautogui.press("esc")

```

```

        return True
    if "escape" in search_query:
        print("Exiting start menu search.")
        pyautogui.press("esc")
        return True
    pyautogui.write(search_query)
    pyautogui.press("enter")
    print(f"Searching and opening: {search_query}")
    time.sleep(5)
    # Get all open window titles
    open_windows = [w.lower() for w in gw.getAllTitles() if w.strip()]
    search_words = search_query.lower().split()
    # Check if any word in the search query appears in any window title
    match_found = any(
        any(word in window for word in search_words) for window in open_windows)
    if match_found:
        print(f"{search_query} opened.")
        return True
    else:
        print("Application not found.")
        pyautogui.press("esc")
        return True
key_map = {
    "new_line": "enter", "backspace": "backspace", "tab": "tab",
    "up": "up", "down": "down", "left": "left", "right": "right",
    "select_all": ("ctrl", "a"), "cut": ("ctrl", "x"),
    "copy": ("ctrl", "c"), "paste": ("ctrl", "v"), "space": "space",
    "delete_word": ("ctrl", "backspace"),
    "delete_line": [("shift", "home"), "backspace"],
    "press_enter": "enter", "press_end": "end"
}
if intent in key_map:
    action = key_map[intent]
    if isinstance(action, tuple):

```

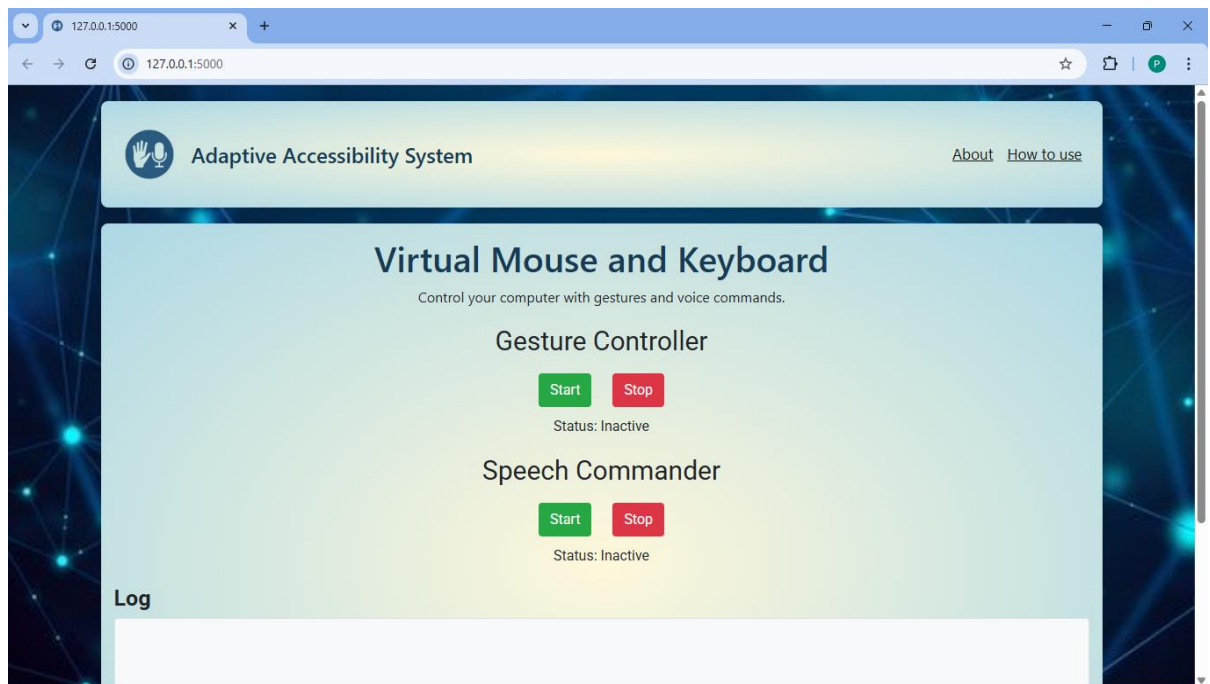
```

        pyautogui.hotkey(*action)
    elif isinstance(action, list):
        for a in action:
            if isinstance(a, tuple):
                pyautogui.hotkey(*a)
            else:
                pyautogui.press(a)
        else:
            pyautogui.press(action)
    print(f"Executed: {intent}")
    return True
print("Unknown command.")
return True

# === MAIN LOOP ===
if __name__ == "__main__":
    print("Voice Assistant is running...")
    paused = False
    while True:
        if paused:
            print("Assistant is paused. Say 'resume assistant' to continue.")
            user_input = recognize_speech()
            if user_input and "resume assistant" in user_input.lower():
                paused = False
                print("Assistant resumed.")
                continue
            user_input = recognize_speech()
        if user_input:
            if "pause assistant" in user_input.lower():
                paused = True
                print("Assistant paused.")
                continue
            if not execute_command(user_input):
                break

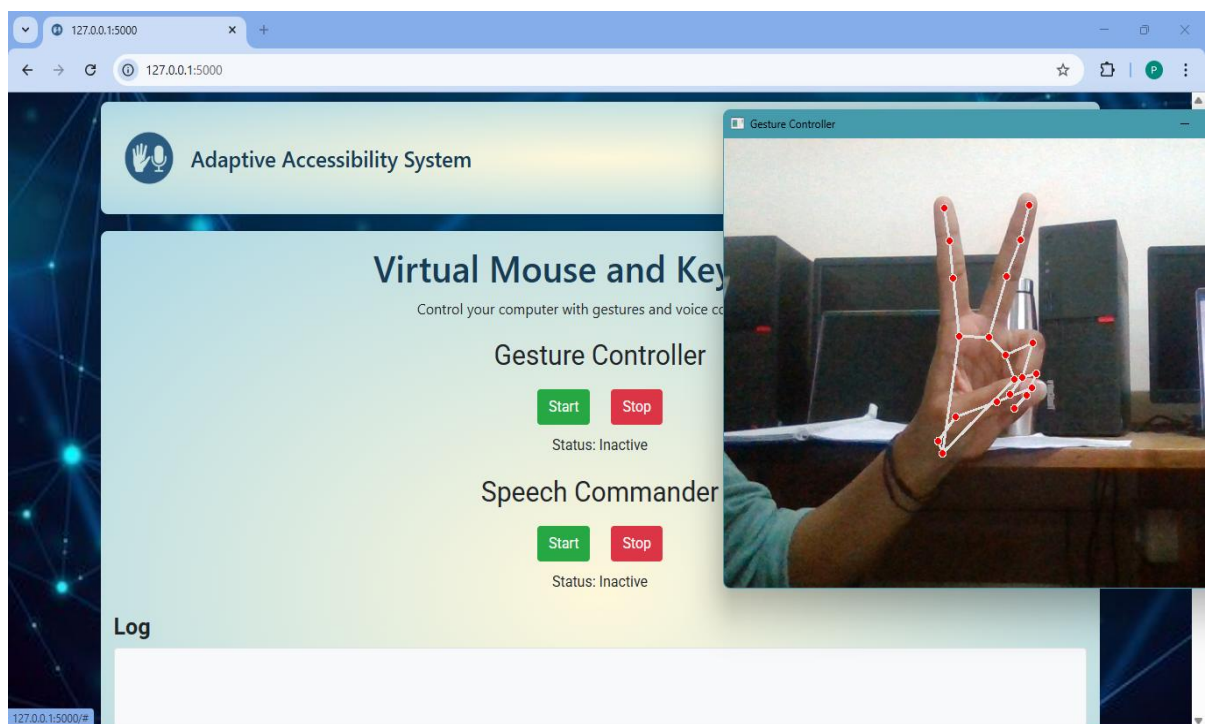
```

5.4 OUTPUT SCREENS



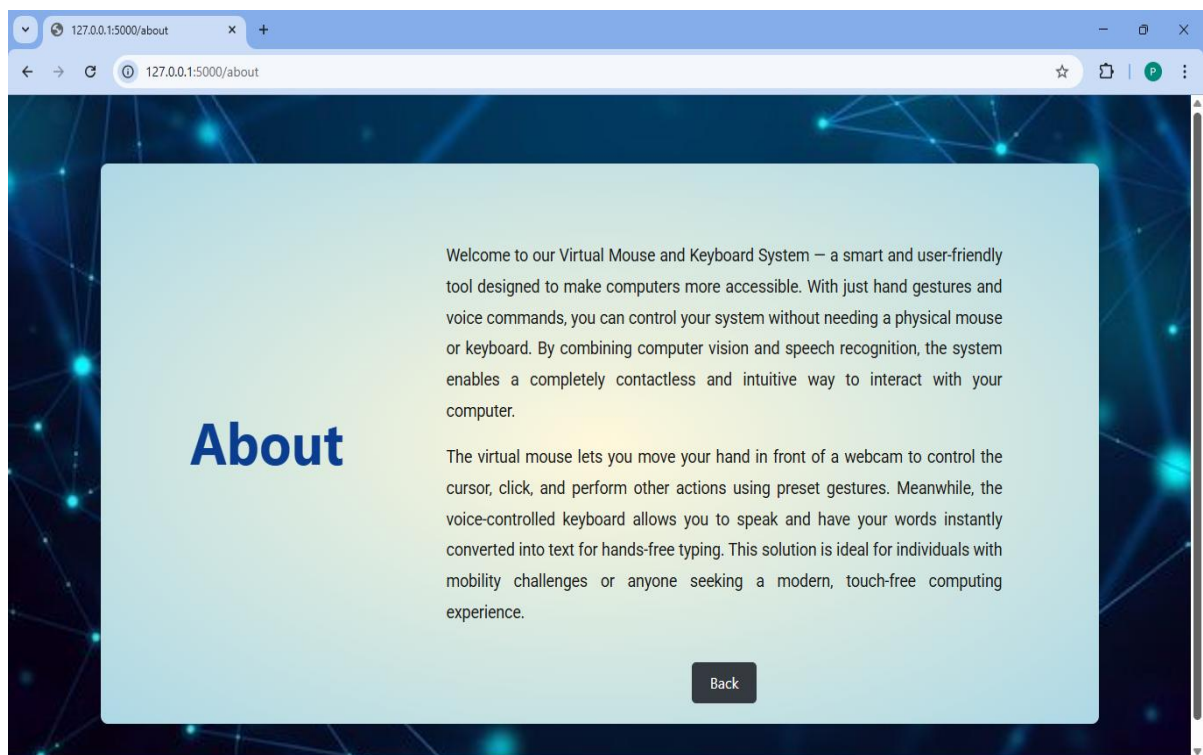
Screen 1. Home Page

Screen 1 displays the home page where the user can start and stop gesture controller and speech commander using the start and stop buttons.



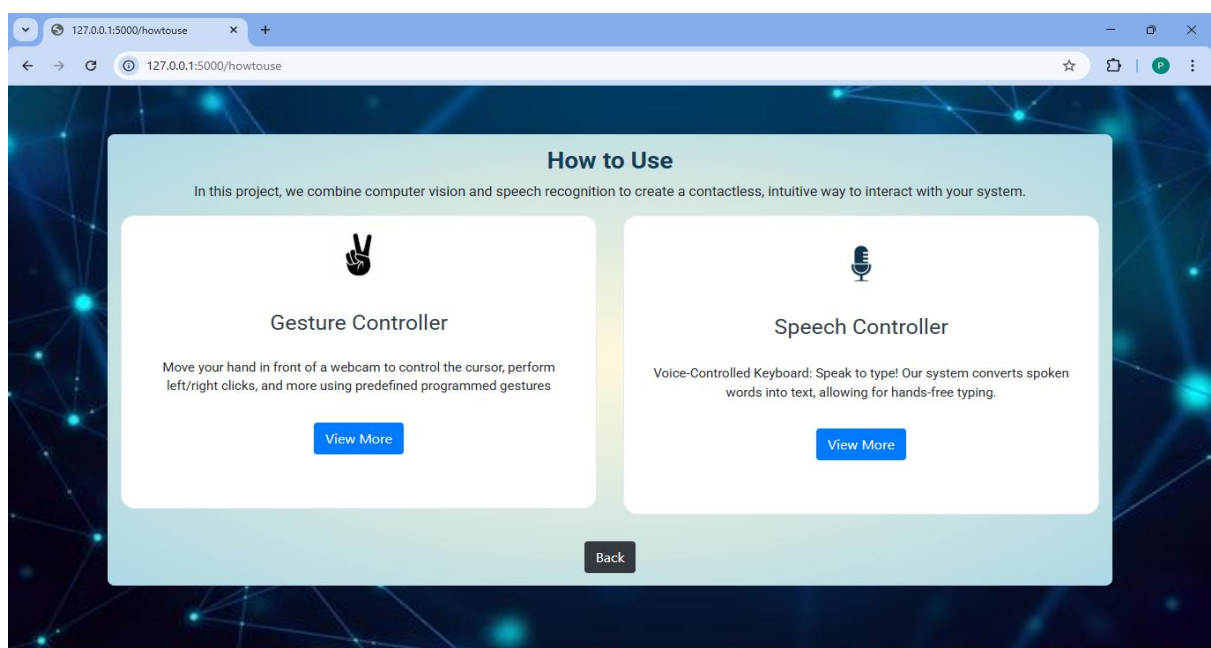
Screen 2. Gesture Controller Started

When the user clicks the start button of gesture controller the camera starts and the user can feed the gestures. The live actions of the user will stream on web log.



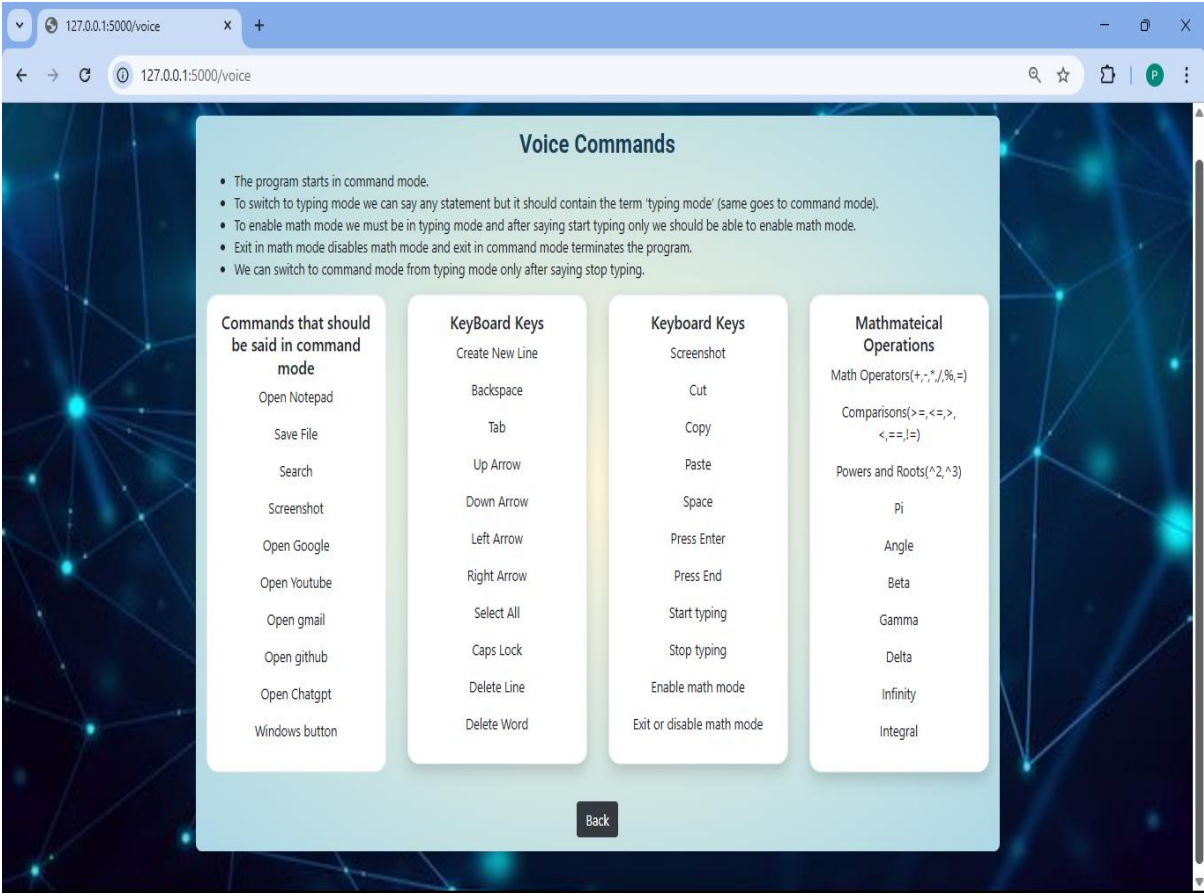
Screen 3. About Page

Screen 3 displays the “About” section of the virtual mouse and keyboard. It explains how the system uses hand gestures and voice commands for user interactions.



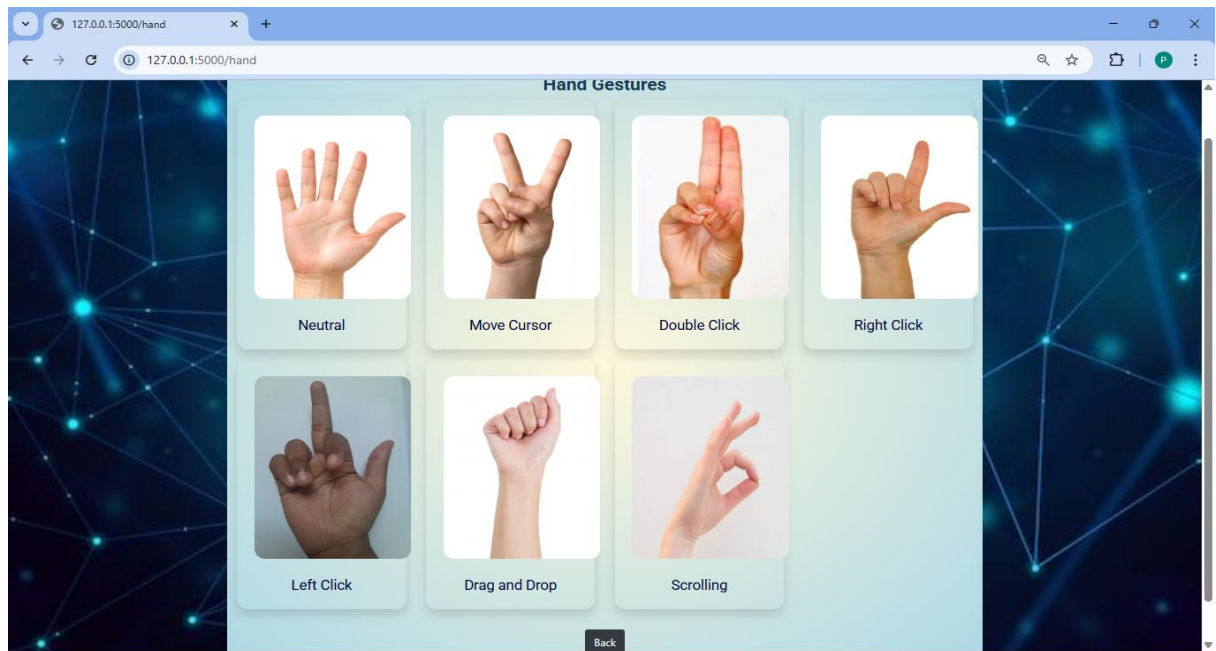
Screen 4. How to use

Screen 4 explains the two main controllers of the system: Gesture and Speech. The Gesture Controller uses hand movements detected by a webcam to operate the cursor and clicks. The Speech Controller enables voice-to-text functionality for hands-free typing.



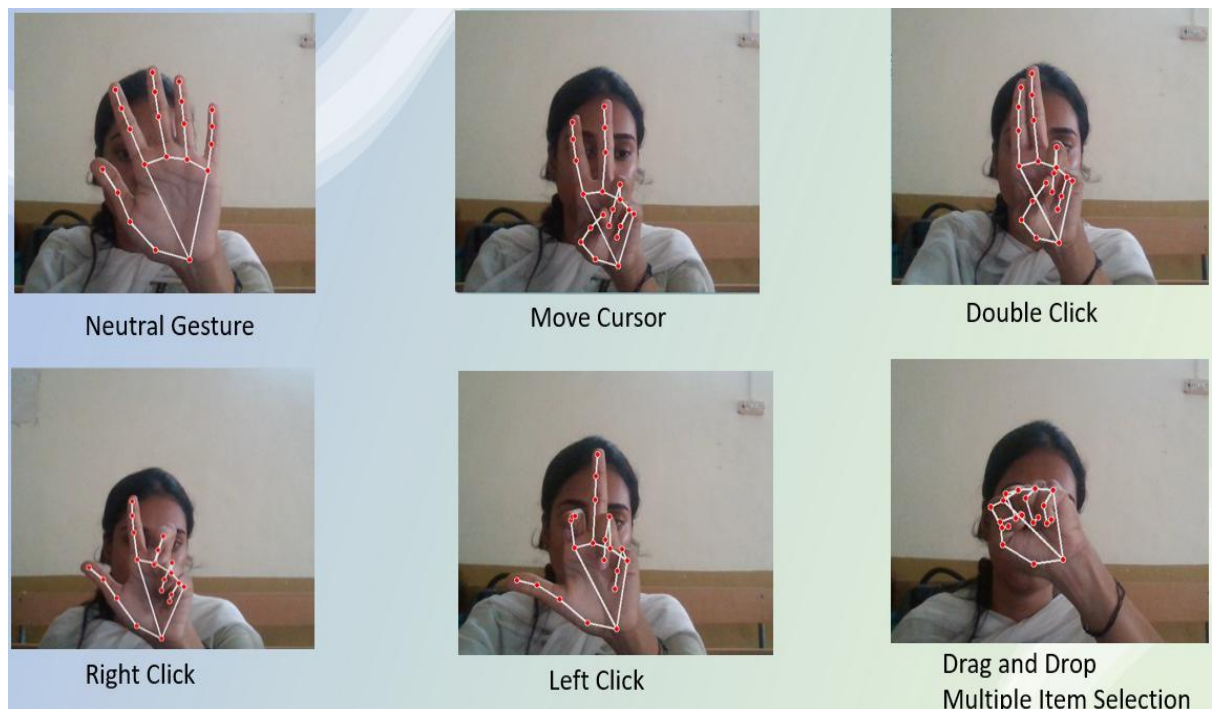
Screen 5. Voice Commands

Screen 5 displays a list of supported voice commands for controlling the keyboard and performing mathematical operations. It helps users understand how to interact using spoken instructions for a hands-free experience.



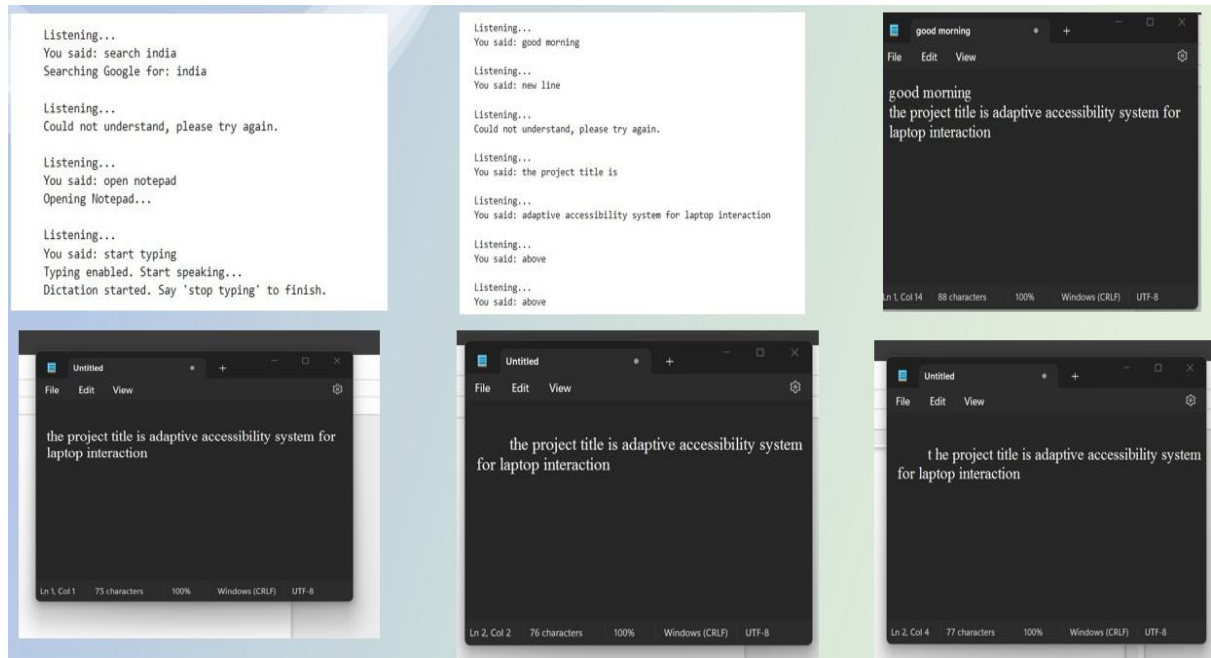
Screen 6. Hand Gestures

Screen 6 showcases the predefined hand gestures used to control the system. Each gesture performs a specific function like cursor movement, clicking, or selection. It visually guides the user in understanding gesture-based controls for seamless navigation.



Screen 7. Detected Gestures

Screen 7 displays real-time detection of hand gestures captured through the webcam. Each gesture is identified and mapped to specific mouse actions like click, drag, or move. It helps visualize how the system interprets physical gestures into functional controls.



Screen 8. Detected Voice Commands

Screen 8 shows the system's response to spoken voice commands converted into text. Voice input is processed and accurately displayed for hands-free keyboard interactions. It demonstrates the effectiveness of speech recognition in real-time.

5.5 CONCLUSION

In this chapter we have discussed the end result part of our project. It discusses some key functions and the introduction to the chapter. It also includes coding and designing part of the application. It includes output screens of the project.

6. CONCLUSION

In conclusion, our project was designed to make using a computer easier and more comfortable by replacing the mouse and keyboard with hand gestures and voice commands. This touch-free system gives people a more natural and simple way to control a computer, which is especially useful for those who have trouble using regular devices or in places like hospitals where keeping things clean is very important. The system lets users move their hands or speak to control the computer without touching anything. While it works well most of the time, it still has a few challenges, like needing good lighting for hand gestures to be seen properly and clear speech for voice commands to work. Even with these small issues, the system has performed well and shows great potential. With some improvements, it could be used in many areas like schools, hospitals, smart homes, and games, making computers easier and more fun for everyone to use.

7. FUTURE SCOPE

In the future, this project can be improved in many ways to make it more useful, easy to use, and accessible for everyone. The hand gesture detection can be made more accurate so that it works well even in different lighting conditions. Voice command recognition can also be improved to understand users better, even when there is background noise. The system can be updated to support more advanced gestures and different languages, which will help people from various regions use it comfortably. Making the system work smoothly on all types of devices, like desktops, laptops, and mobile phones, is another important step. Adding real-time correction for voice input can help by suggesting better options when the system doesn't catch a word correctly, so users won't need to repeat themselves. Creating user profiles that save each person's gesture and voice settings can make the experience more personal and easier. The system can also be connected with other smart devices like TVs, phones, tablets, and home automation tools. Over time, with the help of machine learning, the system can learn from the user's behavior and give smarter and more helpful responses. These improvements will make the technology even more useful in areas like education, healthcare, gaming, smart homes, and workplaces, especially where hands-free control is needed.

8. REFERENCES

- [1] Hax, D. R. T., Penava, P., Krodel, S., Razova, L., & Buettner, R. (2024). A novel hybrid deep learning architecture for dynamic hand gesture recognition. *IEEE Access*, 12, 28761-28774.
- [2] Ch. Akash Reddy, A. Sruthi, G. Hareesh Babu, and A. Balaji (2024), Accessibility System to Control Laptop Without Mouse or Keyboard, Vol. 08, Issue 04, April 2024, DOI: 10.55041/IJSREM31585.
- [3] Kasar, M., Kavimandan, P., Suryawanshi, T., & Abbad, S. (2024). AI-based real-time hand gesture-controlled virtual mouse. *Australian Journal of Electrical and Electronics Engineering*, 21(3), 258-267.
- [4] Pavani, M. G., Priyanka, B., Keerthana, C., Niharika, P., & Prathyusha, V. (2023). Accessibility System to Control Laptop Without Mouse.
- [5] Miah, A. S. M., Hasan, M. A. M., & Shin, J. (2023). Dynamic hand gesture recognition using multi-branch attention based graph and general deep learning model. *IEEE Access*, 11, 4703-4716.
- [6] Park, G., Chandrasegar, V. K., & Koh, J. (2023). Accuracy enhancement of hand gesture recognition using CNN. *IEEE Access*, 11, 26496-26501
- [7] Leon, D. G., Gröli, J., Yeduri, S. R., Rossier, D., Mosqueron, R., Pandey, O. J., & Cenkeramaddi, L. R. (2022). Video hand gestures recognition using depth camera and lightweight cnn. *IEEE Sensors Journal*, 22(14), 14610-14619.
- [8] Dang, T. L., Tran, S. D., Nguyen, T. H., Kim, S., & Monet, N. (2022). An improved hand gesture recognition system using keypoints and hand bounding boxes. *Array*, 16, 100251.
- [9] Jeeru, S., Sivapuram, A. K., León, D. G., Gröli, J., Yeduri, S. R., & Cenkeramaddi, L. R. (2022). Depth camera based dataset of hand gestures. *Data in Brief*, 45, 108659.
- [10] Yuanyuan, S. H. I., Yunan, L. I., Xiaolong, F. U., Miao, K., & Miao, Q. (2021). Review of dynamic gesture recognition. *Virtual Reality & Intelligent Hardware*, 3(3), 183-206.

[11] Tasmere, D., Ahmed, B., & Das, S. R. (2021). Real time hand gesture recognition in depth image using cnn. International Journal of Computer Applications, 174(16), 28-32.

[12] Rahim, M. A., Miah, A. S. M., Sayeed, A., & Shin, J. (2020, August). Hand gesture recognition based on optimal segmentation in human-computer interaction. In 2020 3rd IEEE international conference on knowledge innovation and invention (ICKII) (pp. 163-166). IEEE.

Video Resources: <https://www.youtube.com/watch?v=4NmWNEYtL1s>

<https://www.youtube.com/watch?v=ufm6tfgo-OA&t=107s>

Web Resources: <https://github.com/Viral-Doshi/Gesture-Controlled-Virtual-Mouse>

<https://github.com/Aakriti5/Speech-to-Text/blob/main/TextInfo.mp3>